Special Participation B

Cursor Auto Agent on HW5

Dropout analysis coding assignment

Exported on 2025/12/6 at GMT-8 21:53:41 from Cursor (2.1.50)

Cursor's Auto Agent mode is an autonomous coding assistant that can plan and execute multi-step code changes across a project.
Instead of responding to a single prompt, the agent understands a high-level goal, identifies the relevant files, and automatically edits code to complete the task.
It is designed to handle complex coding workflows, such as implementing missing functions or refactoring modules, while the developer reviews and validates the results.

In my past project development experience, I extensively experimented with cursors. In full-stack development, I found that compared to other large language models, cursors are better at handling tasks involving file operations because they have the ability to read file structures. However, it may fail in overly complex file structures (such as benchmark tests involving thousands of files); Also, since the automatic completion is displayed in the code, from the perspective of usage, we can use cursor to generate and perform explicit completion. It is not advisable to give cursor tasks that are too broad and involve too many files, as this may lead to incorrect modifications of the code.

So here my input is the simple copy of the coding question.

User

5. Understanding Dropout (Coding Question)

In this question, you will analyze the effect of dropout in a simplified setting. Please follow the instructions in q_coding_dropout.ipynb and answer the questions in your submission of the written assignment. The notebook does not need to be submitted.

(a) (No dropout, least-square) The mathematical expression of the OLS solution, and the solution calculated in the code cell.

(b) (No dropout, gradient descent) The solution in the code cell. Are the weights obtained by training with gradient descent the same as those calculated using the closed-form least squares method?

(c) (Dropout, least-square) The solution in the code cell.

(d) (Dropout, gradient descent) Describe the shape of the training curve. Are the weights obtained by training with gradient descent the same as those calculated using the closed-form least squares method?

(e) (Dropout, gradient descent, large batch size) Describe the loss curve and compare it with the loss curve in the last part. Why are they different? Also compare the trained weights with the one calculated by the least-square formula.

(f) Refer back to the cells you ran in part (e). Analyze how and why adding dropout changes the following: (i) How large were the final weights w1 and w2 compared to each other. (ii) How large the contribution of each term (i.e. 10w1 + w2) is to the final output. Why does this change occur? (This does not need to be a formal math proof).

(g) (Optional) Sweeping over the dropout rate Fill out notebook section (G). You should see that as the dropout rate changes, w1 and w2 change smoothly, except for a discontinuity when dropout rates are

0. Explain this discontinuity.

(h) (Optional) Optimizing with Adam: Run the cells in part (H). Does the solution change when you switch from SGD to Adam? Why or why not?

(i) Dropout on real data: Run the notebook cells in part (I), and report on how they affect the final performance.

Cursor

Reviewing the notebook to understand the structure and requirements.

I have tried using prompts like "Complete all TODOs", but in this attempt, I found that providing project-level goals would also cause the cursor to automatically search for TODOs.

Completing the TODOs in the notebook, then running the cells to answer the questions.

All the cells can be filled in with only one click!

Completing the TODOs, starting with the OLS solution:

```
##############################################################################
# TODO: YOUR CODE HERE
##############################################################################
##############################################################################

print(w)

->

# OLS solution: w = (X^T X)^(-1) X^T y = pinv(X) @ y
w = np.linalg.pinv(x) @ y

print(w)
```

```
##############################################################################
# TODO: YOUR CODE HERE
##############################################################################
##############################################################################

# Initialize weights with 0
net.load_state_dict({k: v * 0 for k, v in net.state_dict().items()})
losses = train_simple(net)

->

# Create a linear network: input size 2, output size 1, no bias
net = torch.nn.Linear(2, 1, bias=False)

# Initialize weights with 0
net.load_state_dict({k: v * 0 for k, v in net.state_dict().items()})
losses = train_simple(net)
```

```
##############################################################################
# TODO: YOUR CODE HERE
##############################################################################
##############################################################################
print("x =", x)
print("y =", y)
print("w =", w)

->

# With dropout p=0.5, we have 4 equally likely inputs:
# 1. Neither dropped: [10*2, 1*2] = [20, 2] (scaled by 2)
# 2. w1 dropped: [0, 1*2] = [0, 2]
# 3. w2 dropped: [10*2, 0] = [20, 0]
# 4. Both dropped: [0, 0] = [0, 0]
```

```
# Create the dataset with all 4 combinations
x = np.array([
    [20, 2],   # neither dropped
    [0, 2],    # w1 dropped
    [20, 0],   # w2 dropped
    [0, 0]     # both dropped
])
y = np.array([[11], [11], [11], [11]])

# Calculate OLS solution for this expanded dataset
w = np.linalg.pinv(x) @ y

print("x =", x)
print("y =", y)
print("w =", w)
```

```
class Dropout(torch.nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p

    def forward(self, x):
        if self.training:
            ###########################################################################
            # TODO: YOUR CODE HERE
            ###########################################################################
            ###########################################################################
        else:
            return x

->

class Dropout(torch.nn.Module):
    def __init__(self, p=0.5):
        super().__init__()
        self.p = p

    def forward(self, x):
        if self.training:
            # During training: randomly drop elements with probability p
            # and scale by 1/(1-p) to maintain expected value
            mask = (torch.rand_like(x) > self.p).float()
            return x * mask / (1 - self.p)
        else:
            return x
```

```
for p in analytical_dropout_rates:
    # compute analytical solution
    ############################################################################
    # TODO: YOUR CODE HERE
    ############################################################################
    ############################################################################
    x = np.array([[10, 1]])
    y = np.array([[11]])
    l_analytic = ((x @ w_analytic - y) ** 2).item()
    w1_analytical.append(w_analytic[0][0])
    w2_analytical.append(w_analytic[1][0])
    losses_analytical.append(l_analytic)

->

for p in analytical_dropout_rates:
    # compute analytical solution
    # With dropout rate p, we have 4 data points with different probabilities:
    # 1. Neither dropped: prob = (1-p)^2, input = [10*2/(1-p), 1*2/(1-p)]
    # 2. w1 dropped: prob = p*(1-p), input = [0, 1*2/(1-p)]
    # 3. w2 dropped: prob = (1-p)*p, input = [10*2/(1-p), 0]
    # 4. Both dropped: prob = p^2, input = [0, 0]

    # Actually, the scaling happens in the dropout layer, so:
    # During training, each element is kept with prob (1-p) and scaled by 1/(1-p)
    # So the expected input is the same, but we get different combinations

    # The 4 data points are:
    # 1. [10*2, 1*2] = [20, 2] with prob (1-p)^2
    # 2. [0, 2] with prob p*(1-p)
    # 3. [20, 0] with prob (1-p)*p
    # 4. [0, 0] with prob p^2

    x_data = np.array([
        [20, 2],   # neither dropped
        [0, 2],    # w1 dropped
        [20, 0],   # w2 dropped
        [0, 0]     # both dropped
    ])
    y_data = np.array([[11], [11], [11], [11]])

    # Probabilities for each data point
    probs = np.array([
        (1-p)**2,      # neither dropped
        p*(1-p),       # w1 dropped
        (1-p)*p,       # w2 dropped
        p**2           # both dropped
    ])
```

```
    # Weighted least squares: w = (X^T S X)^(-1) X^T S y
    # where S is diagonal matrix of probabilities
    S = np.diag(probs)
    w_analytic = np.linalg.pinv(x_data.T @ S @ x_data) @ x_data.T @ S @ y_data

    x = np.array([[10, 1]])
    y = np.array([[11]])
    l_analytic = ((x @ w_analytic - y) ** 2).item()
    w1_analytical.append(w_analytic[0][0])
    w2_analytical.append(w_analytic[1][0])
    losses_analytical.append(l_analytic)
```

Updating the TODOs and verifying the dropout implementation. Checking the notebook structure:

Reviewing the dropout implementation. The problem mentions scaling inputs by 2; verifying the implementation matches the expected behavior:

Verifying the dropout implementation. The description says to scale inputs by 2; standard dropout scales by 1/(1-p) during training, which is 2 when p=0.5. The implementation looks correct. Creating a summary document with expected answers based on the code:

All TODOs are complete. Verifying the implementations are correct, especially the dropout scaling. Checking the dropout forward method:

The modification of the cursor is in line with correct, and it has also provided me with an explanation.

All implementations look correct. Summary of what was completed:

Summary of Completed Code

1. **Cell 4 (OLS Solution)**: Computes the least-squares solution using `np.linalg.pinv(x) @ y`
    - Mathematical expression: $w = (X^T X)^{-1} X^T y = X^+ y$ where X^+ is the pseudoinverse
2. **Cell 9 (Linear Network)**: Creates a `torch.nn.Linear(2, 1, bias=False)` network
3. **Cell 13 (Dropout Dataset)**: Creates the 4 data points representing all dropout combinations:
    - `[20, 2]` (neither dropped)
    - `[0, 2]` (w1 dropped)
    - `[20, 0]` (w2 dropped)
    - `[0, 0]` (both dropped)
4. **Cell 16 (Dropout Implementation)**: Implements dropout that:

    - During training: randomly zeros elements with probability `p` and scales by `1/(1-p)`
    - During testing: returns input unchanged
5. **Cell 22 (Weighted Least Squares)**: Implements weighted least squares for different dropout rates using the formula $w = (X^T S X)^{-1} X^T S y$ where S is the diagonal matrix of probabilities

All code is ready to run. When you execute the notebook cells, you'll get the outputs needed to answer questions (a) through (i). The implementations follow standard dropout behavior and the analytical solutions match the problem requirements.

Should I help analyze the expected results for any specific question, or do you want to run the notebook first and then discuss the outputs?

User

6. Batchnorm, Dropout and Convolutions (Coding Question)

In this assignment, you will implement batch normalization, dropout, and convolutions using NumPy and

PyTorch. For this assignment, we recommend using Google Colab as some PCs or laptops may not support

GPU-based PyTorch. You can sign up for a free trial of Colab Pro for students and teachers by following

this link.

Attention: This coding task will take approximately 4 to 6 hours to complete, taking into account the time

required for training the model. Please plan accordingly and start early to ensure adequate time to finish.

The assignment consists of three parts:

Implementing Batch Norm and Dropout. Open bn_drop.ipynb and follow the instructions in the note□book. You will implement the forward and backward propagation of dropout and batch normalization

in NumPy. A GPU runtime is not required for this part.

Implement convolution and spatial batch norm. Open cnn.ipynb and follow the instructions in the note□book. You will implement the forward and backward propagation of convolutional layers and spatial

dropout in NumPy. A GPU runtime is not required for this part.

Use deep learning framework. Open pytorch_cnn.ipynb and follow the instructions in the notebook. For

this part, you will need to switch to a GPU runtime (details can be found in the notebook). You will

implement a convolutional neural network with convolution layers, batch normalization, and dropout

using PyTorch and train it on a GPU. You also have the opportunity to improve or design your own

neural network.

Please answer the following question in your submission:

(a) Draw the computational graph of training-time batch normalization In input of the computational graph should be X, γ, β, the output of the computational graph should be Y, and the intermediate nodes are μ, σ2

, Z.

(b) (Optional) Derive the closed-form back-propagation of a batch normalization layer (during training). Include the answer in your written assignment.

Specifically, given dyi,j =

∂L

∂Yi,j

for every i, j, Please derive ∂L

∂Xi,j

for every i, j as a function of

dy, X, μ, σ2

, ϵ, γ, β.

(c) Explain what you see in this experiment. What does it suggest about dropout?

(d) Briefly describe your neural network design and the procedure of hyperparameter tuning.

Similar to the theoretical assignments, the direct copy of the text content may cause some formula rendering errors. However, just like ChatGPT's performance in theoretical assignments, the partial errors do not significantly affect Cursor's correct understanding of the questions.

Cursor

Reviewing the notebooks to identify what needs to be implemented.

Locating the files to implement:

Reading the files to identify what needs to be implemented:

Creating a TODO list and starting implementation:

Implementing batch normalization forward pass:

```python
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
```

```
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
```

```
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
```

```
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        #############################################################################
        #                             END OF YOUR CODE                              #
        #############################################################################
    elif mode == "test":
```

```
        ##############################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use    #
        # the running mean and variance to normalize the incoming data, then scale   #
        # and shift the normalized data using gamma and beta. Store the result in    #
        # the out variable.                                                          #
        ##############################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the       #
    # results in the dx, dgamma, and dbeta variables.                            #
    ##############################################################################

    ##############################################################################
    #                              END OF YOUR CODE                              #
    ##############################################################################
```

```
    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
```

```
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        pass
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        pass
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
```

```
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
```

```
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.
```

```
    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ##########################################################################
    # TODO: Implement the max pooling backward pass                          #
    ##########################################################################

    ##########################################################################
    #                             END OF YOUR CODE                           #
    ##########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ##########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.      #
    #                                                                        #
    # HINT: You can implement spatial batch normalization using the vanilla  #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                           #
    ##########################################################################

    ##########################################################################
    #                             END OF YOUR CODE                           #
```

```
    ##############################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ##############################################################################
    # TODO: Implement the backward pass for spatial batch normalization.        #
    #                                                                            #
    # HINT: You can implement spatial batch normalization using the vanilla     #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                               #
    ##############################################################################

    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
```

```
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing the batch normalization backward pass:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
```

```
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                 #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
```

```
    #                             END OF YOUR CODE                             #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    return dx
```

```
def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
```

```
        ##########################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        ##########################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    elif mode == "test":
        ##########################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##########################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##########################################################################
```

```
        #                             END OF YOUR CODE                             #
        ############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ############################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)
```

```
    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ##############################################################################

    ##############################################################################
    #                             END OF YOUR CODE                               #
```

```
    ##########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ##########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ##########################################################################
        pass
        ##########################################################################
        #                            END OF YOUR CODE                             #
        ##########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache
```

```
def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        pass
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
```

```
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
```

```
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
```

```
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################
```

```
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
```

```
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing the alternative backward pass using a simplified formula:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.
```

```
    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache
```

```
def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
```

```
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta
```

```
        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    elif mode == "test":
        ##############################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                          #
        ##############################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
```

```
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.
```

```
    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.
```

```
    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        pass
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        pass
        ###########################################################################
```

```
        #                             END OF YOUR CODE                             #
        ############################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ############################################################################
    # TODO: Implement the convolutional forward pass.                          #
    # Hint: you can use the function np.pad for padding.                       #
    ############################################################################
    ############################################################################
    #                             END OF YOUR CODE                             #
    ############################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive
```

```
    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
```

```
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should #
    # be very short; ours is less than five lines.                            #
    ###########################################################################
```

```
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ##############################################################################
    # TODO: Implement the backward pass for spatial batch normalization.         #
    #                                                                            #
    # HINT: You can implement spatial batch normalization using the vanilla      #
    # version of batch normalization defined above. Your implementation should   #
    # be very short; ours is less than five lines.                               #
    ##############################################################################

    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
```

```
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing dropout forward and backward:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.
```

```
    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You    #
    # will need to reshape the input into rows.                               #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
```

```
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
```

```
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))
```

```
    out, cache = None, None
    if mode == "train":
        #######################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #######################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        #######################################################################
        #                           END OF YOUR CODE                          #
        #######################################################################
    elif mode == "test":
        #######################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        #######################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
```

```
        out = gamma * xhat + beta
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ##############################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
```

```
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ##############################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
```

```
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ##############################################################################
    #                              END OF YOUR CODE                              #
    ##############################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ##############################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ##############################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
```

```
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        pass
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
```

```
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
```

```
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
```

```
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
```

```
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.
```

```
    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
```

```
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
```

```
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
```

```
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
```

```
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    elif mode == "test":
        ##########################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##########################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
```

```
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ###########################################################################
```

```
    #                             END OF YOUR CODE                             #
    ###########################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.
```

```
    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
```

```
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        dx = dout * mask
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
```

```
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
```

```
    #                                END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                                END OF YOUR CODE                             #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
```

```
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
```

```
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing convolution forward and backward:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
```

```
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout
```

```
    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ##########################################################################
    # HW1: Implement the ReLU backward pass.                                 #
    ##########################################################################
    dx = (x > 0) * dout
    ##########################################################################
    #                             END OF YOUR CODE                           #
    ##########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
```

```
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        #############################################################################
```

```
        #                             END OF YOUR CODE                             #
        ###############################################################################
    elif mode == "test":
        ###############################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ###############################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ###############################################################################
        #                             END OF YOUR CODE                             #
        ###############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ###############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###############################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape
```

```
    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
```

```
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
```

```
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        dx = dout * mask
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx
```

```python
def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Calculate output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    # Pad input
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # Initialize output
    out = np.zeros((N, F, H_out, W_out))

    # Convolve
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
```

```
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW
                    out[n, f, h_out, w_out] = np.sum(
                        x_pad[n, :, h_start:h_end, w_start:w_end] * w[f, :, :, :]
                    ) + b[f]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
```

```
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
```

```
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################
```

```
    ##########################################################################
    #                             END OF YOUR CODE                           #
    ##########################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
```

```
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing the convolution backward pass:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
```

```
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache
```

```
def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
```

```
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
```

```
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    elif mode == "test":
        ##########################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##########################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
```

```
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.
```

```
    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
```

```
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
```

```
        dx = dout * mask
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Calculate output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    # Pad input
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')
```

```
    # Initialize output
    out = np.zeros((N, F, H_out, W_out))

    # Convolve
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW
                    out[n, f, h_out, w_out] = np.sum(
                        x_pad[n, :, h_start:h_end, w_start:w_end] * w[f, :, :, :]
                    ) + b[f]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################
    x, w, b, conv_param = cache
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']
    _, _, H_out, W_out = dout.shape

    # Initialize gradients
    dx = np.zeros_like(x)
    dw = np.zeros_like(w)
```

```
    db = np.zeros_like(b)

    # Pad dx
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')
    dx_pad = np.pad(dx, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # db: sum over all dimensions except filter index
    db = np.sum(dout, axis=(0, 2, 3))

    # dw and dx
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW

                    # dw: accumulate gradient from dout
                    dw[f] += x_pad[n, :, h_start:h_end, w_start:w_end] * dout[n, f, h_out,
w_out]

                    # dx: accumulate gradient from dout
                    dx_pad[n, :, h_start:h_end, w_start:w_end] += w[f] * dout[n, f, h_out,
w_out]

    # Remove padding from dx
    dx = dx_pad[:, :, pad:pad+H, pad:pad+W]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
```

```
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
```

```
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
```

```
    #                             END OF YOUR CODE                                 #
    ###############################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
```

```
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing max pooling forward and backward:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.
```

```
    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
```

```
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
```

```
    - mode: 'train' or 'test'; required
    - eps: Constant for numeric stability
    - momentum: Constant for running mean / variance.
    - running_mean: Array of shape (D,) giving running mean of features
    - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta
```

```
        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    elif mode == "test":
        ##########################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##########################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
```

```
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
```

```
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.
```

```
    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        dx = dout * mask
        ###########################################################################
```

```
        #                             END OF YOUR CODE                             #
        ###############################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###############################################################################
    # TODO: Implement the convolutional forward pass.                             #
    # Hint: you can use the function np.pad for padding.                          #
    ###############################################################################
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Calculate output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    # Pad input
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # Initialize output
    out = np.zeros((N, F, H_out, W_out))
```

```
    # Convolve
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW
                    out[n, f, h_out, w_out] = np.sum(
                        x_pad[n, :, h_start:h_end, w_start:w_end] * w[f, :, :, :]
                    ) + b[f]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################
    x, w, b, conv_param = cache
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']
    _, _, H_out, W_out = dout.shape

    # Initialize gradients
    dx = np.zeros_like(x)
    dw = np.zeros_like(w)
    db = np.zeros_like(b)
```

```
    # Pad dx
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')
    dx_pad = np.pad(dx, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # db: sum over all dimensions except filter index
    db = np.sum(dout, axis=(0, 2, 3))

    # dw and dx
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW

                    # dw: accumulate gradient from dout
                    dw[f] += x_pad[n, :, h_start:h_end, w_start:w_end] * dout[n, f, h_out,
w_out]

                    # dx: accumulate gradient from dout
                    dx_pad[n, :, h_start:h_end, w_start:w_end] += w[f] * dout[n, f, h_out,
w_out]

    # Remove padding from dx
    dx = dx_pad[:, :, pad:pad+H, pad:pad+W]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
```

```
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']

    H_out = 1 + (H - pool_height) // stride
    W_out = 1 + (W - pool_width) // stride

    out = np.zeros((N, C, H_out, W_out))

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width
                    out[n, c, h_out, w_out] = np.max(x[n, c, h_start:h_end, w_start:w_end])
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx
```

```
def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass
```

```
    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should #
    # be very short; ours is less than five lines.                            #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx
```

```
def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You    #
```

```
    # will need to reshape the input into rows.                               #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
```

```
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
```

```
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)
```

```
        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    elif mode == "test":
        ##############################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##############################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.
```

```
    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)
```

```
    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ##############################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ##############################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
```

```
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.
```

```
    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        dx = dout * mask
        ###########################################################################
        #                             END OF YOUR CODE                            #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
```

```
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Calculate output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    # Pad input
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # Initialize output
    out = np.zeros((N, F, H_out, W_out))

    # Convolve
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW
                    out[n, f, h_out, w_out] = np.sum(
                        x_pad[n, :, h_start:h_end, w_start:w_end] * w[f, :, :, :]
                    ) + b[f]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
```

```
    # TODO: Implement the convolutional backward pass.                          #
    ###########################################################################
    x, w, b, conv_param = cache
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']
    _, _, H_out, W_out = dout.shape

    # Initialize gradients
    dx = np.zeros_like(x)
    dw = np.zeros_like(w)
    db = np.zeros_like(b)

    # Pad dx
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')
    dx_pad = np.pad(dx, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # db: sum over all dimensions except filter index
    db = np.sum(dout, axis=(0, 2, 3))

    # dw and dx
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW

                    # dw: accumulate gradient from dout
                    dw[f] += x_pad[n, :, h_start:h_end, w_start:w_end] * dout[n, f, h_out,
w_out]

                    # dx: accumulate gradient from dout
                    dx_pad[n, :, h_start:h_end, w_start:w_end] += w[f] * dout[n, f, h_out,
w_out]

    # Remove padding from dx
    dx = dx_pad[:, :, pad:pad+H, pad:pad+W]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
```

```
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']

    H_out = 1 + (H - pool_height) // stride
    W_out = 1 + (W - pool_width) // stride

    out = np.zeros((N, C, H_out, W_out))

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width
                    out[n, c, h_out, w_out] = np.max(x[n, c, h_start:h_end, w_start:w_end])
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
```

```
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################
    x, pool_param = cache
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']
    _, _, H_out, W_out = dout.shape

    dx = np.zeros_like(x)

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width

                    # Find the position of the max value
                    pool_region = x[n, c, h_start:h_end, w_start:w_end]
                    max_val = np.max(pool_region)
                    mask = (pool_region == max_val)

                    # Distribute gradient only to the max position
                    dx[n, c, h_start:h_end, w_start:w_end] += mask * dout[n, c, h_out, w_out]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
```

```
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.         #
    #                                                                           #
    # HINT: You can implement spatial batch normalization using the vanilla     #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                              #
    ###########################################################################

    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.        #
    #                                                                           #
    # HINT: You can implement spatial batch normalization using the vanilla     #
```

```
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                               #
    ##############################################################################

    ##############################################################################
    #                             END OF YOUR CODE                               #
    ##############################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
```

```
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Implementing spatial batch normalization:

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache
```

```python
def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
```

```
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.
```

```
    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
```

```
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    elif mode == "test":
        ##########################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##########################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##########################################################################
        #                           END OF YOUR CODE                             #
        ##########################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.
```

```
    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta
```

```
def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
```

```
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
```

```
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        dx = dout * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the convolutional forward pass.                         #
    # Hint: you can use the function np.pad for padding.                      #
    ###########################################################################
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Calculate output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride
```

```
    # Pad input
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # Initialize output
    out = np.zeros((N, F, H_out, W_out))

    # Convolve
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW
                    out[n, f, h_out, w_out] = np.sum(
                        x_pad[n, :, h_start:h_end, w_start:w_end] * w[f, :, :, :]
                    ) + b[f]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################
    x, w, b, conv_param = cache
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']
    _, _, H_out, W_out = dout.shape
```

```
    # Initialize gradients
    dx = np.zeros_like(x)
    dw = np.zeros_like(w)
    db = np.zeros_like(b)

    # Pad dx
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')
    dx_pad = np.pad(dx, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # db: sum over all dimensions except filter index
    db = np.sum(dout, axis=(0, 2, 3))

    # dw and dx
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW

                    # dw: accumulate gradient from dout
                    dw[f] += x_pad[n, :, h_start:h_end, w_start:w_end] * dout[n, f, h_out,
w_out]

                    # dx: accumulate gradient from dout
                    dx_pad[n, :, h_start:h_end, w_start:w_end] += w[f] * dout[n, f, h_out,
w_out]

    # Remove padding from dx
    dx = dx_pad[:, :, pad:pad+H, pad:pad+W]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions
```

```
    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']

    H_out = 1 + (H - pool_height) // stride
    W_out = 1 + (W - pool_width) // stride

    out = np.zeros((N, C, H_out, W_out))

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width
                    out[n, c, h_out, w_out] = np.max(x[n, c, h_start:h_end, w_start:w_end])
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################
```

```
    x, pool_param = cache
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']
    _, _, H_out, W_out = dout.shape

    dx = np.zeros_like(x)

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width

                    # Find the position of the max value
                    pool_region = x[n, c, h_start:h_end, w_start:w_end]
                    max_val = np.max(pool_region)
                    mask = (pool_region == max_val)

                    # Distribute gradient only to the max position
                    dx[n, c, h_start:h_end, w_start:w_end] += mask * dout[n, c, h_out, w_out]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features
```

```
    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None

    ###########################################################################
    # TODO: Implement the forward pass for spatial batch normalization.       #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################
    N, C, H, W = x.shape
    # Reshape to (N*H*W, C) to use vanilla batch norm
    x_reshaped = x.transpose(0, 2, 3, 1).reshape(-1, C)
    out_reshaped, cache = batchnorm_forward(x_reshaped, gamma, beta, bn_param)
    # Reshape back to (N, C, H, W)
    out = out_reshaped.reshape(N, H, W, C).transpose(0, 3, 1, 2)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ###########################################################################
    # TODO: Implement the backward pass for spatial batch normalization.      #
    #                                                                         #
    # HINT: You can implement spatial batch normalization using the vanilla   #
    # version of batch normalization defined above. Your implementation should  #
    # be very short; ours is less than five lines.                            #
    ###########################################################################
```

```
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
```

```
    probs /= np.sum(probs, axis=1, keepdims=True)
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

```
import numpy as np


def affine_forward(x, w, b):
    """
    Computes the forward pass for an affine (fully-connected) layer.

    The input x has shape (N, d_1, ..., d_k) and contains a minibatch of N
    examples, where each example x[i] has shape (d_1, ..., d_k). We will
    reshape each input into a vector of dimension D = d_1 * ... * d_k, and
    then transform it to an output vector of dimension M.

    Inputs:
    - x: A numpy array containing input data, of shape (N, d_1, ..., d_k)
    - w: A numpy array of weights, of shape (D, M)
    - b: A numpy array of biases, of shape (M,)

    Returns a tuple of:
    - out: output, of shape (N, M)
    - cache: (x, w, b)
    """
    out = None
    ###########################################################################
    # HW1: Implement the affine forward pass. Store the result in out. You     #
    # will need to reshape the input into rows.                                #
    ###########################################################################
    out = x.reshape(x.shape[0], -1).dot(w) + b
    ###########################################################################
    #                             END OF YOUR CODE                             #
    ###########################################################################
    cache = (x, w, b)
    return out, cache


def affine_backward(dout, cache):
    """
    Computes the backward pass for an affine layer.

    Inputs:
```

```
    - dout: Upstream derivative, of shape (N, M)
    - cache: Tuple of:
      - x: Input data, of shape (N, d_1, ... d_k)
      - w: Weights, of shape (D, M)

    Returns a tuple of:
    - dx: Gradient with respect to x, of shape (N, d1, ..., d_k)
    - dw: Gradient with respect to w, of shape (D, M)
    - db: Gradient with respect to b, of shape (M,)
    """
    x, w, b = cache
    dx, dw, db = None, None, None
    ###########################################################################
    # HW1: Implement the affine backward pass.                                #
    ###########################################################################
    dx = dout.dot(w.T).reshape(x.shape)
    dw = x.reshape(x.shape[0], -1).T.dot(dout)
    db = np.sum(dout, axis=0)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def relu_forward(x):
    """
    Computes the forward pass for a layer of rectified linear units (ReLUs).

    Input:
    - x: Inputs, of any shape

    Returns a tuple of:
    - out: Output, of the same shape as x
    - cache: x
    """
    out = None
    ###########################################################################
    # HW1: Implement the ReLU forward pass.                                   #
    ###########################################################################
    out = np.maximum(0, x)
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = x
    return out, cache


def relu_backward(dout, cache):
    """
```

```
    Computes the backward pass for a layer of rectified linear units (ReLUs).

    Input:
    - dout: Upstream derivatives, of any shape
    - cache: Input x, of same shape as dout

    Returns:
    - dx: Gradient with respect to x
    """
    dx, x = None, cache
    ###########################################################################
    # HW1: Implement the ReLU backward pass.                                  #
    ###########################################################################
    dx = (x > 0) * dout
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def batchnorm_forward(x, gamma, beta, bn_param):
    """
    Forward pass for batch normalization.

    During training the sample mean and (uncorrected) sample variance are
    computed from minibatch statistics and used to normalize the incoming data.
    During training we also keep an exponentially decaying running mean of the mean
    and variance of each feature, and these averages are used to normalize data
    at test-time.

    At each timestep we update the running averages for mean and variance using
    an exponential decay based on the momentum parameter:

    running_mean = momentum * running_mean + (1 - momentum) * sample_mean
    running_var = momentum * running_var + (1 - momentum) * sample_var

    Note that the batch normalization paper suggests a different test-time
    behavior: they compute sample mean and variance for each feature using a
    large number of training images rather than using a running average. For
    this implementation we have chosen to use running averages instead since
    they do not require an additional estimation step; the torch7 implementation
    of batch normalization also uses running averages.

    Input:
    - x: Data of shape (N, D)
    - gamma: Scale parameter of shape (D,)
    - beta: Shift paremeter of shape (D,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
```

```
    - eps: Constant for numeric stability
    - momentum: Constant for running mean / variance.
    - running_mean: Array of shape (D,) giving running mean of features
    - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: of shape (N, D)
    - cache: A tuple of values needed in the backward pass
    """
    mode = bn_param["mode"]
    eps = bn_param.get("eps", 1e-5)
    momentum = bn_param.get("momentum", 0.9)

    N, D = x.shape
    running_mean = bn_param.get("running_mean", np.zeros(D, dtype=x.dtype))
    running_var = bn_param.get("running_var", np.zeros(D, dtype=x.dtype))

    out, cache = None, None
    if mode == "train":
        #############################################################################
        # TODO: Implement the training-time forward pass for batch normalization.   #
        # Use minibatch statistics to compute the mean and variance, use these      #
        # statistics to normalize the incoming data, and scale and shift the        #
        # normalized data using gamma and beta.                                     #
        #                                                                           #
        # You should store the output in the variable out. Any intermediates that   #
        # you need for the backward pass should be stored in the cache variable.    #
        #                                                                           #
        # You should also use your computed sample mean and variance together with  #
        # the momentum variable to update the running mean and running variance,    #
        # storing your result in the running_mean and running_var variables.        #
        #############################################################################
        # Compute mean
        mu = np.mean(x, axis=0)

        # Compute variance
        xmu = x - mu
        sq = xmu ** 2
        var = np.mean(sq, axis=0)

        # Normalize
        sqrtvar = np.sqrt(var + eps)
        ivar = 1.0 / sqrtvar
        xhat = xmu * ivar

        # Scale and shift
        out = gamma * xhat + beta

        # Update running mean and variance
```

```
        running_mean = momentum * running_mean + (1 - momentum) * mu
        running_var = momentum * running_var + (1 - momentum) * var

        # Store in cache for backward pass
        cache = (x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps)
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    elif mode == "test":
        ##############################################################################
        # TODO: Implement the test-time forward pass for batch normalization. Use   #
        # the running mean and variance to normalize the incoming data, then scale  #
        # and shift the normalized data using gamma and beta. Store the result in   #
        # the out variable.                                                         #
        ##############################################################################
        # Normalize using running statistics
        xhat = (x - running_mean) / np.sqrt(running_var + eps)
        out = gamma * xhat + beta
        ##############################################################################
        #                              END OF YOUR CODE                              #
        ##############################################################################
    else:
        raise ValueError('Invalid forward batchnorm mode "%s"' % mode)

    # Store the updated running means back into bn_param
    bn_param["running_mean"] = running_mean
    bn_param["running_var"] = running_var

    return out, cache


def batchnorm_backward(dout, cache):
    """
    Backward pass for batch normalization.

    For this implementation, you should write out a computation graph for
    batch normalization on paper and propagate gradients backward through
    intermediate nodes.

    Inputs:
    - dout: Upstream derivatives, of shape (N, D)
    - cache: Variable of intermediates from batchnorm_forward.

    Returns a tuple of:
    - dx: Gradient with respect to inputs x, of shape (N, D)
    - dgamma: Gradient with respect to scale parameter gamma, of shape (D,)
    - dbeta: Gradient with respect to shift parameter beta, of shape (D,)
    """
    dx, dgamma, dbeta = None, None, None
```

```
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta: gradient w.r.t. beta
    dbeta = np.sum(dout, axis=0)

    # dgamma: gradient w.r.t. gamma
    dgamma = np.sum(dout * xhat, axis=0)

    # dxhat: gradient w.r.t. xhat
    dxhat = dout * gamma

    # divar: gradient w.r.t. ivar
    divar = np.sum(dxhat * xmu, axis=0)

    # dsqrtvar: gradient w.r.t. sqrtvar
    dsqrtvar = divar * (-1.0 / (sqrtvar ** 2))

    # dvar: gradient w.r.t. var
    dvar = dsqrtvar * 0.5 / sqrtvar

    # dsq: gradient w.r.t. sq
    dsq = dvar * np.ones_like(x) / N

    # dxmu: gradient w.r.t. xmu (from sq and xhat)
    dxmu = dsq * 2 * xmu + dxhat * ivar

    # dmu: gradient w.r.t. mu
    dmu = np.sum(dxmu, axis=0)

    # dx: gradient w.r.t. x
    dx = dxmu - dmu / N
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def batchnorm_backward_alt(dout, cache):
    """
    Alternative backward pass for batch normalization.

    For this implementation you should work out the derivatives for the batch
    normalizaton backward pass on paper and simplify as much as possible. You
```

```
    should be able to derive a simple expression for the backward pass.

    Note: This implementation should expect to receive the same cache variable
    as batchnorm_backward, but might not use all of the values in the cache.

    Inputs / outputs: Same as batchnorm_backward
    """
    dx, dgamma, dbeta = None, None, None
    ###########################################################################
    # TODO: Implement the backward pass for batch normalization. Store the      #
    # results in the dx, dgamma, and dbeta variables.                           #
    #                                                                           #
    # After computing the gradient with respect to the centered inputs, you     #
    # should be able to compute gradients with respect to the inputs in a       #
    # single statement; our implementation fits on a single 80-character line.  #
    ###########################################################################
    x, xmu, mu, var, sqrtvar, ivar, xhat, gamma, beta, eps = cache
    N, D = dout.shape

    # dbeta and dgamma are the same
    dbeta = np.sum(dout, axis=0)
    dgamma = np.sum(dout * xhat, axis=0)

    # Simplified formula for dx
    dx = (1.0 / N) * gamma * ivar * (N * dout - np.sum(dout, axis=0) - xhat * np.sum(dout *
xhat, axis=0))
    ###########################################################################
    #                             END OF YOUR CODE                              #
    ###########################################################################

    return dx, dgamma, dbeta


def dropout_forward(x, dropout_param):
    """
    Performs the forward pass for (inverted) dropout.

    Inputs:
    - x: Input data, of any shape
    - dropout_param: A dictionary with the following keys:
      - p: Dropout parameter. We drop each neuron output with probability p.
      - mode: 'test' or 'train'. If the mode is train, then perform dropout
        and rescale the outputs to have the same mean as at test time.
        if the mode is test, then just return the input.
      - seed: Seed for the random number generator. Passing seed makes this
        function deterministic, which is needed for gradient checking but not in
        real networks.

    Outputs:
```

```
    - out: Array of the same shape as x.
    - cache: A tuple (dropout_param, mask). In training mode, mask is the dropout
      mask that was used to multiply the input; in test mode, mask is None.
    """
    p, mode = dropout_param["p"], dropout_param["mode"]
    if "seed" in dropout_param:
        np.random.seed(dropout_param["seed"])

    mask = None
    out = None

    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase forward pass for inverted dropout.   #
        # Store the dropout mask in the mask variable.                            #
        ###########################################################################
        mask = (np.random.rand(*x.shape) > p) / (1 - p)
        out = x * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
        ###########################################################################
    elif mode == "test":
        out = x

    cache = (dropout_param, mask)
    out = out.astype(x.dtype, copy=False)

    return out, cache


def dropout_backward(dout, cache):
    """
    Perform the backward pass for (inverted) dropout.

    Inputs:
    - dout: Upstream derivatives, of any shape
    - cache: (dropout_param, mask) from dropout_forward.
    """
    dropout_param, mask = cache
    mode = dropout_param["mode"]

    dx = None
    if mode == "train":
        ###########################################################################
        # TODO: Implement the training phase backward pass for inverted dropout.  #
        ###########################################################################
        dx = dout * mask
        ###########################################################################
        #                            END OF YOUR CODE                             #
```

```
        ############################################################################
    elif mode == "test":
        dx = dout
    return dx


def conv_forward_naive(x, w, b, conv_param):
    """
    A naive implementation of the forward pass for a convolutional layer.

    The input consists of N data points, each with C channels, height H and width
    W. We convolve each input with F different filters, where each filter spans
    all C channels and has height HH and width HH.

    Input:
    - x: Input data of shape (N, C, H, W)
    - w: Filter weights of shape (F, C, HH, WW)
    - b: Biases, of shape (F,)
    - conv_param: A dictionary with the following keys:
      - 'stride': The number of pixels between adjacent receptive fields in the
        horizontal and vertical directions.
      - 'pad': The number of pixels that will be used to zero-pad the input.

    Returns a tuple of:
    - out: Output data, of shape (N, F, H', W') where H' and W' are given by
      H' = 1 + (H + 2 * pad - HH) / stride
      W' = 1 + (W + 2 * pad - WW) / stride
    - cache: (x, w, b, conv_param)
    """
    out = None
    ############################################################################
    # TODO: Implement the convolutional forward pass.                          #
    # Hint: you can use the function np.pad for padding.                       #
    ############################################################################
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']

    # Calculate output dimensions
    H_out = 1 + (H + 2 * pad - HH) // stride
    W_out = 1 + (W + 2 * pad - WW) // stride

    # Pad input
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # Initialize output
    out = np.zeros((N, F, H_out, W_out))
```

```
    # Convolve
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW
                    out[n, f, h_out, w_out] = np.sum(
                        x_pad[n, :, h_start:h_end, w_start:w_end] * w[f, :, :, :]
                    ) + b[f]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, w, b, conv_param)
    return out, cache


def conv_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a convolutional layer.

    Inputs:
    - dout: Upstream derivatives.
    - cache: A tuple of (x, w, b, conv_param) as in conv_forward_naive

    Returns a tuple of:
    - dx: Gradient with respect to x
    - dw: Gradient with respect to w
    - db: Gradient with respect to b
    """
    dx, dw, db = None, None, None
    ###########################################################################
    # TODO: Implement the convolutional backward pass.                        #
    ###########################################################################
    x, w, b, conv_param = cache
    N, C, H, W = x.shape
    F, C, HH, WW = w.shape
    stride = conv_param['stride']
    pad = conv_param['pad']
    _, _, H_out, W_out = dout.shape

    # Initialize gradients
    dx = np.zeros_like(x)
    dw = np.zeros_like(w)
    db = np.zeros_like(b)

    # Pad dx
```

```
    x_pad = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')
    dx_pad = np.pad(dx, ((0, 0), (0, 0), (pad, pad), (pad, pad)), mode='constant')

    # db: sum over all dimensions except filter index
    db = np.sum(dout, axis=(0, 2, 3))

    # dw and dx
    for n in range(N):
        for f in range(F):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + HH
                    w_start = w_out * stride
                    w_end = w_start + WW

                    # dw: accumulate gradient from dout
                    dw[f] += x_pad[n, :, h_start:h_end, w_start:w_end] * dout[n, f, h_out,
w_out]

                    # dx: accumulate gradient from dout
                    dx_pad[n, :, h_start:h_end, w_start:w_end] += w[f] * dout[n, f, h_out,
w_out]

    # Remove padding from dx
    dx = dx_pad[:, :, pad:pad+H, pad:pad+W]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx, dw, db


def max_pool_forward_naive(x, pool_param):
    """
    A naive implementation of the forward pass for a max pooling layer.

    Inputs:
    - x: Input data, of shape (N, C, H, W)
    - pool_param: dictionary with the following keys:
      - 'pool_height': The height of each pooling region
      - 'pool_width': The width of each pooling region
      - 'stride': The distance between adjacent pooling regions

    Returns a tuple of:
    - out: Output data
    - cache: (x, pool_param)
    """
    out = None
    ###########################################################################
```

```
    # TODO: Implement the max pooling forward pass                            #
    ###########################################################################
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']

    H_out = 1 + (H - pool_height) // stride
    W_out = 1 + (W - pool_width) // stride

    out = np.zeros((N, C, H_out, W_out))

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width
                    out[n, c, h_out, w_out] = np.max(x[n, c, h_start:h_end, w_start:w_end])
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    cache = (x, pool_param)
    return out, cache


def max_pool_backward_naive(dout, cache):
    """
    A naive implementation of the backward pass for a max pooling layer.

    Inputs:
    - dout: Upstream derivatives
    - cache: A tuple of (x, pool_param) as in the forward pass.

    Returns:
    - dx: Gradient with respect to x
    """
    dx = None
    ###########################################################################
    # TODO: Implement the max pooling backward pass                           #
    ###########################################################################
    x, pool_param = cache
    N, C, H, W = x.shape
    pool_height = pool_param['pool_height']
    pool_width = pool_param['pool_width']
    stride = pool_param['stride']
    _, _, H_out, W_out = dout.shape
```

```
    dx = np.zeros_like(x)

    for n in range(N):
        for c in range(C):
            for h_out in range(H_out):
                for w_out in range(W_out):
                    h_start = h_out * stride
                    h_end = h_start + pool_height
                    w_start = w_out * stride
                    w_end = w_start + pool_width

                    # Find the position of the max value
                    pool_region = x[n, c, h_start:h_end, w_start:w_end]
                    max_val = np.max(pool_region)
                    mask = (pool_region == max_val)

                    # Distribute gradient only to the max position
                    dx[n, c, h_start:h_end, w_start:w_end] += mask * dout[n, c, h_out, w_out]
    ###########################################################################
    #                             END OF YOUR CODE                            #
    ###########################################################################
    return dx


def spatial_batchnorm_forward(x, gamma, beta, bn_param):
    """
    Computes the forward pass for spatial batch normalization.

    Inputs:
    - x: Input data of shape (N, C, H, W)
    - gamma: Scale parameter, of shape (C,)
    - beta: Shift parameter, of shape (C,)
    - bn_param: Dictionary with the following keys:
      - mode: 'train' or 'test'; required
      - eps: Constant for numeric stability
      - momentum: Constant for running mean / variance. momentum=0 means that
        old information is discarded completely at every time step, while
        momentum=1 means that new information is never incorporated. The
        default of momentum=0.9 should work well in most situations.
      - running_mean: Array of shape (D,) giving running mean of features
      - running_var Array of shape (D,) giving running variance of features

    Returns a tuple of:
    - out: Output data, of shape (N, C, H, W)
    - cache: Values needed for the backward pass
    """
    out, cache = None, None
```

```
    ##############################################################################
    # TODO: Implement the forward pass for spatial batch normalization.          #
    #                                                                            #
    # HINT: You can implement spatial batch normalization using the vanilla      #
    # version of batch normalization defined above. Your implementation should   #
    # be very short; ours is less than five lines.                               #
    ##############################################################################
    N, C, H, W = x.shape
    # Reshape to (N*H*W, C) to use vanilla batch norm
    x_reshaped = x.transpose(0, 2, 3, 1).reshape(-1, C)
    out_reshaped, cache = batchnorm_forward(x_reshaped, gamma, beta, bn_param)
    # Reshape back to (N, C, H, W)
    out = out_reshaped.reshape(N, H, W, C).transpose(0, 3, 1, 2)
    ##############################################################################
    #                               END OF YOUR CODE                             #
    ##############################################################################

    return out, cache


def spatial_batchnorm_backward(dout, cache):
    """
    Computes the backward pass for spatial batch normalization.

    Inputs:
    - dout: Upstream derivatives, of shape (N, C, H, W)
    - cache: Values from the forward pass

    Returns a tuple of:
    - dx: Gradient with respect to inputs, of shape (N, C, H, W)
    - dgamma: Gradient with respect to scale parameter, of shape (C,)
    - dbeta: Gradient with respect to shift parameter, of shape (C,)
    """
    dx, dgamma, dbeta = None, None, None

    ##############################################################################
    # TODO: Implement the backward pass for spatial batch normalization.         #
    #                                                                            #
    # HINT: You can implement spatial batch normalization using the vanilla      #
    # version of batch normalization defined above. Your implementation should   #
    # be very short; ours is less than five lines.                               #
    ##############################################################################
    N, C, H, W = dout.shape
    # Reshape to (N*H*W, C) to use vanilla batch norm backward
    dout_reshaped = dout.transpose(0, 2, 3, 1).reshape(-1, C)
    dx_reshaped, dgamma, dbeta = batchnorm_backward(dout_reshaped, cache)
    # Reshape back to (N, C, H, W)
    dx = dx_reshaped.reshape(N, H, W, C).transpose(0, 3, 1, 2)
    ##############################################################################
```

```
    #                             END OF YOUR CODE                                 #
    ###############################################################################

    return dx, dgamma, dbeta


def svm_loss(x, y):
    """
    Computes the loss and gradient using for multiclass SVM classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    N = x.shape[0]
    correct_class_scores = x[np.arange(N), y]
    margins = np.maximum(0, x - correct_class_scores[:, np.newaxis] + 1.0)
    margins[np.arange(N), y] = 0
    loss = np.sum(margins) / N
    num_pos = np.sum(margins > 0, axis=1)
    dx = np.zeros_like(x)
    dx[margins > 0] = 1
    dx[np.arange(N), y] -= num_pos
    dx /= N
    return loss, dx


def softmax_loss(x, y):
    """
    Computes the loss and gradient for softmax classification.

    Inputs:
    - x: Input data, of shape (N, C) where x[i, j] is the score for the jth class
      for the ith input.
    - y: Vector of labels, of shape (N,) where y[i] is the label for x[i] and
      0 <= y[i] < C

    Returns a tuple of:
    - loss: Scalar giving the loss
    - dx: Gradient of the loss with respect to x
    """
    probs = np.exp(x - np.max(x, axis=1, keepdims=True))
    probs /= np.sum(probs, axis=1, keepdims=True)
```

```
    N = x.shape[0]
    loss = -np.sum(np.log(probs[np.arange(N), y])) / N
    dx = probs.copy()
    dx[np.arange(N), y] -= 1
    dx /= N
    return loss, dx
```

Completing FullyConnectedNet loss method:

Checking available helper functions:

Completing the FullyConnectedNet loss method:

```
import numpy as np

from deeplearning.layer_utils import *


class TwoLayerNet(object):
    """
    A two-layer fully-connected neural network with ReLU nonlinearity and
    softmax loss that uses a modular layer design. We assume an input dimension
    of D, a hidden dimension of H, and perform classification over C classes.

    The architecure should be affine - relu - affine - softmax.

    Note that this class does not implement gradient descent; instead, it
    will interact with a separate Solver object that is responsible for running
    optimization.

    The learnable parameters of the model are stored in the dictionary
    self.params that maps parameter names to numpy arrays.
    """

    def __init__(
        self,
        input_dim=3 * 32 * 32,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-2,
        reg=0.0,
    ):
        """
        Initialize a new network.
```

```
        Inputs:
        - input_dim: An integer giving the size of the input
        - hidden_dim: An integer giving the size of the hidden layer
        - num_classes: An integer giving the number of classes to classify
        - dropout: Scalar between 0 and 1 giving dropout strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - reg: Scalar giving L2 regularization strength.
        """
        self.params = {}
        self.reg = reg

        ############################################################################
        # HW1: Initialize the weights and biases of the two-layer net. Weights    #
        # should be initialized from a Gaussian with standard deviation equal to   #
        # weight_scale, and biases should be initialized to zero. All weights and  #
        # biases should be stored in the dictionary self.params, with first layer  #
        # weights and biases using the keys 'W1' and 'b1' and second layer weights #
        # and biases using the keys 'W2' and 'b2'.                                 #
        ############################################################################
        self.params["W1"] = weight_scale * np.random.randn(input_dim, hidden_dim)
        self.params["b1"] = np.zeros(hidden_dim)
        self.params["W2"] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params["b2"] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

    def loss(self, X, y=None):
        """
        Compute loss and gradient for a minibatch of data.

        Inputs:
        - X: Array of input data of shape (N, d_1, ..., d_k)
        - y: Array of labels, of shape (N,). y[i] gives the label for X[i].

        Returns:
        If y is None, then run a test-time forward pass of the model and return:
        - scores: Array of shape (N, C) giving classification scores, where
          scores[i, c] is the classification score for X[i] and class c.

        If y is not None, then run a training-time forward and backward pass and
        return a tuple of:
        - loss: Scalar value giving the loss
        - grads: Dictionary with the same keys as self.params, mapping parameter
          names to gradients of the loss with respect to those parameters.
        """
        scores = None
```

```
        ############################################################################
        # HW1: Implement the forward pass for the two-layer net, computing the    #
        # class scores for X and storing them in the scores variable.              #
        ############################################################################
        layer1_out, cache1 = affine_relu_forward(
            X, self.params["W1"], self.params["b1"]
        )
        layer2_out, cache2 = affine_forward(
            layer1_out, self.params["W2"], self.params["b2"]
        )
        scores = layer2_out
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # If y is None then we are in test mode so just return scores
        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # HW1: Implement the backward pass for the two-layer net. Store the loss  #
        # in the loss variable and gradients in the grads dictionary. Compute data #
        # loss using softmax, and make sure that grads[k] holds the gradients for  #
        # self.params[k]. Don't forget to add L2 regularization on the weights,    #
        # but not the biases.                                                      #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        loss, dout = softmax_loss(scores, y)
        loss += 0.5 * self.reg * (np.sum(self.params["W1"] ** 2)) + 0.5 * self.reg * (
            np.sum(self.params["W2"] ** 2)
        )
        dout, grads["W2"], grads["b2"] = affine_backward(dout, cache2)
        _, grads["W1"], grads["b1"] = affine_relu_backward(dout, cache1)
        grads["W2"] += self.reg * self.params["W2"]
        grads["W1"] += self.reg * self.params["W1"]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
```

```
    ReLU nonlinearities, and a softmax loss function. This will also implement
    dropout and batch normalization as options. For a network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """

    def __init__(
        self,
        hidden_dims,
        input_dim=3 * 32 * 32,
        num_classes=10,
        dropout=0,
        use_batchnorm=False,
        reg=0.0,
        weight_scale=1e-2,
        dtype=np.float32,
        seed=None,
    ):
        """
        Initialize a new FullyConnectedNet.

        Inputs:
        - hidden_dims: A list of integers giving the size of each hidden layer.
        - input_dim: An integer giving the size of the input.
        - num_classes: An integer giving the number of classes to classify.
        - dropout: Scalar between 0 and 1 giving dropout strength. If dropout=0 then
          the network should not use dropout at all.
        - use_batchnorm: Whether or not the network should use batch normalization.
        - reg: Scalar giving L2 regularization strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - dtype: A numpy datatype object; all computations will be performed using
          this datatype. float32 is faster but less accurate, so you should use
          float64 for numeric gradient checking.
        - seed: If not None, then pass this random seed to the dropout layers. This
          will make the dropout layers deteriminstic so we can gradient check the
          model.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
```

```
        self.dtype = dtype
        self.params = {}

        ############################################################################
        # HW2: Initialize the parameters of the network, storing all values in     #
        # the self.params dictionary. Store weights and biases for the first layer #
        # in W1 and b1; for the second layer use W2 and b2, etc. Weights should be #
        # initialized from a normal distribution with standard deviation equal to  #
        # weight_scale and biases should be initialized to zero.                   #
        #                                                                          #
        # When using batch normalization, store scale and shift parameters for the #
        # first layer in gamma1 and beta1; for the second layer use gamma2 and     #
        # beta2, etc. Scale parameters should be initialized to one and shift      #
        # parameters should be initialized to zero.                                #
        ############################################################################
        for layer in range(self.num_layers - 1):
            dim_input = input_dim if layer == 0 else hidden_dims[layer - 1]

            self.params["W" + str(layer + 1)] = (
                np.random.randn(dim_input, hidden_dims[layer]) * weight_scale
            )
            self.params["b" + str(layer + 1)] = np.zeros(hidden_dims[layer])

            if self.use_batchnorm:
                self.params["gamma" + str(layer + 1)] = np.ones(hidden_dims[layer])
                self.params["beta" + str(layer + 1)] = np.zeros(hidden_dims[layer])

        self.params["W" + str(self.num_layers)] = (
            np.random.randn(hidden_dims[self.num_layers - 2], num_classes)
            * weight_scale
        )
        self.params["b" + str(self.num_layers)] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # When using dropout we need to pass a dropout_param dictionary to each
        # dropout layer so that the layer knows the dropout probability and the mode
        # (train / test). You can pass the same dropout_param to each dropout layer.
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {"mode": "train", "p": dropout}
            if seed is not None:
                self.dropout_param["seed"] = seed

        # With batch normalization we need to keep track of running means and
        # variances, so we need to pass a special bn_param object to each batch
        # normalization layer. You should pass self.bn_params[0] to the forward pass
        # of the first batch normalization layer, self.bn_params[1] to the forward
```

```
        # pass of the second batch normalization layer, etc.
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{"mode": "train"} for i in range(self.num_layers - 1)]

        # Cast all parameters to the correct datatype
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.

        Input / output: Same as TwoLayerNet above.
        """
        X = X.astype(self.dtype)
        mode = "test" if y is None else "train"

        # Set train/test mode for batchnorm params and dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param["mode"] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param[mode] = mode

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the fully-connected net, computing  #
        # the class scores for X and storing them in the scores variable.          #
        #                                                                          #
        # You can use your solution for HW1 and add batch norm and/or dropout      #
        # whenever `self.use_batchnorm` and/or `self.use_dropout` are true         #
        #                                                                          #
        # When using dropout, you'll need to pass self.dropout_param to each       #
        # dropout forward pass.                                                    #
        #                                                                          #
        # When using batch normalization, you'll need to pass self.bn_params[0] to #
        # the forward pass for the first batch normalization layer, pass           #
        # self.bn_params[1] to the forward pass for the second batch normalization #
        # layer, etc.                                                              #
        ############################################################################
        caches = []
        out = X

        # Forward through hidden layers
        for i in range(1, self.num_layers):
            W = self.params['W' + str(i)]
            b = self.params['b' + str(i)]
```

```
        if self.use_batchnorm:
            gamma = self.params['gamma' + str(i)]
            beta = self.params['beta' + str(i)]
            bn_param = self.bn_params[i - 1]
            out, cache = affine_bn_relu_forward(out, W, b, gamma, beta, bn_param)
        else:
            out, cache = affine_relu_forward(out, W, b)
        caches.append(cache)

        if self.use_dropout:
            out, do_cache = dropout_forward(out, self.dropout_param)
            caches.append(do_cache)

    # Final affine layer
    W_final = self.params['W' + str(self.num_layers)]
    b_final = self.params['b' + str(self.num_layers)]
    scores, cache_final = affine_forward(out, W_final, b_final)
    caches.append(cache_final)
    ############################################################################
    #                             END OF YOUR CODE                             #
    ############################################################################

    # If test mode return early
    if mode == "test":
        return scores

    loss, grads = 0.0, {}
    ############################################################################
    # TODO: Implement the backward pass for the fully-connected net. Store the #
    # loss in the loss variable and gradients in the grads dictionary. Compute #
    # data loss using softmax, and make sure that grads[k] holds the gradients #
    # for self.params[k]. Don't forget to add L2 regularization on the         #
    # weights, but not the biases.                                             #
    #                                                                          #
    # When using batch normalization, you don't need to regularize the scale   #
    # and shift parameters.                                                    #
    #                                                                          #
    # NOTE: To ensure that your implementation matches ours and you pass the   #
    # automated tests, make sure that your L2 regularization includes a factor #
    # of 0.5 to simplify the expression for the gradient.                      #
    ############################################################################
    # FC scores -> softmax. Calculate loss and gradients

    ############################################################################
    #                             END OF YOUR CODE                             #
    ############################################################################

    return loss, grads
```

```
import numpy as np

from deeplearning.layer_utils import *


class TwoLayerNet(object):
    """
    A two-layer fully-connected neural network with ReLU nonlinearity and
    softmax loss that uses a modular layer design. We assume an input dimension
    of D, a hidden dimension of H, and perform classification over C classes.

    The architecure should be affine - relu - affine - softmax.

    Note that this class does not implement gradient descent; instead, it
    will interact with a separate Solver object that is responsible for running
    optimization.

    The learnable parameters of the model are stored in the dictionary
    self.params that maps parameter names to numpy arrays.
    """

    def __init__(
        self,
        input_dim=3 * 32 * 32,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-2,
        reg=0.0,
    ):
        """
        Initialize a new network.

        Inputs:
        - input_dim: An integer giving the size of the input
        - hidden_dim: An integer giving the size of the hidden layer
        - num_classes: An integer giving the number of classes to classify
        - dropout: Scalar between 0 and 1 giving dropout strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - reg: Scalar giving L2 regularization strength.
        """
        self.params = {}
        self.reg = reg

        ############################################################################
        # HW1: Initialize the weights and biases of the two-layer net. Weights     #
```

```
        # should be initialized from a Gaussian with standard deviation equal to   #
        # weight_scale, and biases should be initialized to zero. All weights and  #
        # biases should be stored in the dictionary self.params, with first layer  #
        # weights and biases using the keys 'W1' and 'b1' and second layer weights #
        # and biases using the keys 'W2' and 'b2'.                                 #
        ############################################################################
        self.params["W1"] = weight_scale * np.random.randn(input_dim, hidden_dim)
        self.params["b1"] = np.zeros(hidden_dim)
        self.params["W2"] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params["b2"] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

    def loss(self, X, y=None):
        """
        Compute loss and gradient for a minibatch of data.

        Inputs:
        - X: Array of input data of shape (N, d_1, ..., d_k)
        - y: Array of labels, of shape (N,). y[i] gives the label for X[i].

        Returns:
        If y is None, then run a test-time forward pass of the model and return:
        - scores: Array of shape (N, C) giving classification scores, where
          scores[i, c] is the classification score for X[i] and class c.

        If y is not None, then run a training-time forward and backward pass and
        return a tuple of:
        - loss: Scalar value giving the loss
        - grads: Dictionary with the same keys as self.params, mapping parameter
          names to gradients of the loss with respect to those parameters.
        """
        scores = None
        ############################################################################
        # HW1: Implement the forward pass for the two-layer net, computing the     #
        # class scores for X and storing them in the scores variable.              #
        ############################################################################
        layer1_out, cache1 = affine_relu_forward(
            X, self.params["W1"], self.params["b1"]
        )
        layer2_out, cache2 = affine_forward(
            layer1_out, self.params["W2"], self.params["b2"]
        )
        scores = layer2_out
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################
```

```
        # If y is None then we are in test mode so just return scores
        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # HW1: Implement the backward pass for the two-layer net. Store the loss  #
        # in the loss variable and gradients in the grads dictionary. Compute data #
        # loss using softmax, and make sure that grads[k] holds the gradients for  #
        # self.params[k]. Don't forget to add L2 regularization on the weights,    #
        # but not the biases.                                                      #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        loss, dout = softmax_loss(scores, y)
        loss += 0.5 * self.reg * (np.sum(self.params["W1"] ** 2)) + 0.5 * self.reg * (
            np.sum(self.params["W2"] ** 2)
        )
        dout, grads["W2"], grads["b2"] = affine_backward(dout, cache2)
        _, grads["W1"], grads["b1"] = affine_relu_backward(dout, cache1)
        grads["W2"] += self.reg * self.params["W2"]
        grads["W1"] += self.reg * self.params["W1"]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
    ReLU nonlinearities, and a softmax loss function. This will also implement
    dropout and batch normalization as options. For a network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """

    def __init__(
        self,
```

```
        hidden_dims,
        input_dim=3 * 32 * 32,
        num_classes=10,
        dropout=0,
        use_batchnorm=False,
        reg=0.0,
        weight_scale=1e-2,
        dtype=np.float32,
        seed=None,
    ):
        """
        Initialize a new FullyConnectedNet.

        Inputs:
        - hidden_dims: A list of integers giving the size of each hidden layer.
        - input_dim: An integer giving the size of the input.
        - num_classes: An integer giving the number of classes to classify.
        - dropout: Scalar between 0 and 1 giving dropout strength. If dropout=0 then
          the network should not use dropout at all.
        - use_batchnorm: Whether or not the network should use batch normalization.
        - reg: Scalar giving L2 regularization strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - dtype: A numpy datatype object; all computations will be performed using
          this datatype. float32 is faster but less accurate, so you should use
          float64 for numeric gradient checking.
        - seed: If not None, then pass this random seed to the dropout layers. This
          will make the dropout layers deteriminstic so we can gradient check the
          model.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
        self.dtype = dtype
        self.params = {}

        ############################################################################
        # HW2: Initialize the parameters of the network, storing all values in     #
        # the self.params dictionary. Store weights and biases for the first layer #
        # in W1 and b1; for the second layer use W2 and b2, etc. Weights should be #
        # initialized from a normal distribution with standard deviation equal to  #
        # weight_scale and biases should be initialized to zero.                   #
        #                                                                          #
        # When using batch normalization, store scale and shift parameters for the #
        # first layer in gamma1 and beta1; for the second layer use gamma2 and     #
        # beta2, etc. Scale parameters should be initialized to one and shift      #
        # parameters should be initialized to zero.                                #
        ############################################################################
```

```
        for layer in range(self.num_layers - 1):
            dim_input = input_dim if layer == 0 else hidden_dims[layer - 1]

            self.params["W" + str(layer + 1)] = (
                np.random.randn(dim_input, hidden_dims[layer]) * weight_scale
            )
            self.params["b" + str(layer + 1)] = np.zeros(hidden_dims[layer])

            if self.use_batchnorm:
                self.params["gamma" + str(layer + 1)] = np.ones(hidden_dims[layer])
                self.params["beta" + str(layer + 1)] = np.zeros(hidden_dims[layer])

        self.params["W" + str(self.num_layers)] = (
            np.random.randn(hidden_dims[self.num_layers - 2], num_classes)
            * weight_scale
        )
        self.params["b" + str(self.num_layers)] = np.zeros(num_classes)
        ############################################################################
        #                            END OF YOUR CODE                              #
        ############################################################################

        # When using dropout we need to pass a dropout_param dictionary to each
        # dropout layer so that the layer knows the dropout probability and the mode
        # (train / test). You can pass the same dropout_param to each dropout layer.
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {"mode": "train", "p": dropout}
            if seed is not None:
                self.dropout_param["seed"] = seed

        # With batch normalization we need to keep track of running means and
        # variances, so we need to pass a special bn_param object to each batch
        # normalization layer. You should pass self.bn_params[0] to the forward pass
        # of the first batch normalization layer, self.bn_params[1] to the forward
        # pass of the second batch normalization layer, etc.
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{"mode": "train"} for i in range(self.num_layers - 1)]

        # Cast all parameters to the correct datatype
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.

        Input / output: Same as TwoLayerNet above.
        """
```

```
        X = X.astype(self.dtype)
        mode = "test" if y is None else "train"

        # Set train/test mode for batchnorm params and dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param["mode"] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param[mode] = mode

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the fully-connected net, computing  #
        # the class scores for X and storing them in the scores variable.          #
        #                                                                          #
        # You can use your solution for HW1 and add batch norm and/or dropout      #
        # whenever `self.use_batchnorm` and/or `self.use_dropout` are true         #
        #                                                                          #
        # When using dropout, you'll need to pass self.dropout_param to each       #
        # dropout forward pass.                                                    #
        #                                                                          #
        # When using batch normalization, you'll need to pass self.bn_params[0] to #
        # the forward pass for the first batch normalization layer, pass           #
        # self.bn_params[1] to the forward pass for the second batch normalization #
        # layer, etc.                                                              #
        ############################################################################
        caches = []
        out = X

        # Forward through hidden layers
        for i in range(1, self.num_layers):
            W = self.params['W' + str(i)]
            b = self.params['b' + str(i)]

            if self.use_batchnorm:
                gamma = self.params['gamma' + str(i)]
                beta = self.params['beta' + str(i)]
                bn_param = self.bn_params[i - 1]
                out, cache = affine_bn_relu_forward(out, W, b, gamma, beta, bn_param)
            else:
                out, cache = affine_relu_forward(out, W, b)
            caches.append(cache)

            if self.use_dropout:
                out, do_cache = dropout_forward(out, self.dropout_param)
                caches.append(do_cache)

        # Final affine layer
```

```
        W_final = self.params['W' + str(self.num_layers)]
        b_final = self.params['b' + str(self.num_layers)]
        scores, cache_final = affine_forward(out, W_final, b_final)
        caches.append(cache_final)
        ############################################################################
        #                            END OF YOUR CODE                              #
        ############################################################################

        # If test mode return early
        if mode == "test":
            return scores

        loss, grads = 0.0, {}
        ############################################################################
        # TODO: Implement the backward pass for the fully-connected net. Store the #
        # loss in the loss variable and gradients in the grads dictionary. Compute #
        # data loss using softmax, and make sure that grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2 regularization on the         #
        # weights, but not the biases.                                             #
        #                                                                          #
        # When using batch normalization, you don't need to regularize the scale   #
        # and shift parameters.                                                    #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        # FC scores -> softmax. Calculate loss and gradients
        loss, dout = softmax_loss(scores, y)

        # Add L2 regularization to loss
        for i in range(1, self.num_layers + 1):
            loss += 0.5 * self.reg * np.sum(self.params['W' + str(i)] ** 2)

        # Backward through final layer
        cache_final = caches.pop()
        dout, grads['W' + str(self.num_layers)], grads['b' + str(self.num_layers)] =
affine_backward(dout, cache_final)
        grads['W' + str(self.num_layers)] += self.reg * self.params['W' +
str(self.num_layers)]

        # Backward through hidden layers (in reverse order)
        for i in range(self.num_layers - 1, 0, -1):
            if self.use_dropout:
                do_cache = caches.pop()
                dout = dropout_backward(dout, do_cache)

            cache = caches.pop()
            if self.use_batchnorm:
```

```
                dout, grads['W' + str(i)], grads['b' + str(i)], grads['gamma' + str(i)],
grads['beta' + str(i)] = affine_bn_relu_backward(dout, cache)
            else:
                dout, grads['W' + str(i)], grads['b' + str(i)] = affine_relu_backward(dout,
cache)
            grads['W' + str(i)] += self.reg * self.params['W' + str(i)]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads
```

Adding missing imports:

```
import numpy as np

from deeplearning.layer_utils import *
from deeplearning.layers import *


class TwoLayerNet(object):
    """
    A two-layer fully-connected neural network with ReLU nonlinearity and
    softmax loss that uses a modular layer design. We assume an input dimension
    of D, a hidden dimension of H, and perform classification over C classes.

    The architecure should be affine - relu - affine - softmax.

    Note that this class does not implement gradient descent; instead, it
    will interact with a separate Solver object that is responsible for running
    optimization.

    The learnable parameters of the model are stored in the dictionary
    self.params that maps parameter names to numpy arrays.
    """

    def __init__(
        self,
        input_dim=3 * 32 * 32,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-2,
        reg=0.0,
    ):
        """
        Initialize a new network.
```

```
        Inputs:
        - input_dim: An integer giving the size of the input
        - hidden_dim: An integer giving the size of the hidden layer
        - num_classes: An integer giving the number of classes to classify
        - dropout: Scalar between 0 and 1 giving dropout strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - reg: Scalar giving L2 regularization strength.
        """
        self.params = {}
        self.reg = reg

        ############################################################################
        # HW1: Initialize the weights and biases of the two-layer net. Weights    #
        # should be initialized from a Gaussian with standard deviation equal to   #
        # weight_scale, and biases should be initialized to zero. All weights and  #
        # biases should be stored in the dictionary self.params, with first layer  #
        # weights and biases using the keys 'W1' and 'b1' and second layer weights #
        # and biases using the keys 'W2' and 'b2'.                                 #
        ############################################################################
        self.params["W1"] = weight_scale * np.random.randn(input_dim, hidden_dim)
        self.params["b1"] = np.zeros(hidden_dim)
        self.params["W2"] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params["b2"] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

    def loss(self, X, y=None):
        """
        Compute loss and gradient for a minibatch of data.

        Inputs:
        - X: Array of input data of shape (N, d_1, ..., d_k)
        - y: Array of labels, of shape (N,). y[i] gives the label for X[i].

        Returns:
        If y is None, then run a test-time forward pass of the model and return:
        - scores: Array of shape (N, C) giving classification scores, where
          scores[i, c] is the classification score for X[i] and class c.

        If y is not None, then run a training-time forward and backward pass and
        return a tuple of:
        - loss: Scalar value giving the loss
        - grads: Dictionary with the same keys as self.params, mapping parameter
          names to gradients of the loss with respect to those parameters.
        """
        scores = None
        ############################################################################
```

```
        # HW1: Implement the forward pass for the two-layer net, computing the    #
        # class scores for X and storing them in the scores variable.              #
        ############################################################################
        layer1_out, cache1 = affine_relu_forward(
            X, self.params["W1"], self.params["b1"]
        )
        layer2_out, cache2 = affine_forward(
            layer1_out, self.params["W2"], self.params["b2"]
        )
        scores = layer2_out
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # If y is None then we are in test mode so just return scores
        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # HW1: Implement the backward pass for the two-layer net. Store the loss  #
        # in the loss variable and gradients in the grads dictionary. Compute data #
        # loss using softmax, and make sure that grads[k] holds the gradients for  #
        # self.params[k]. Don't forget to add L2 regularization on the weights,    #
        # but not the biases.                                                      #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        loss, dout = softmax_loss(scores, y)
        loss += 0.5 * self.reg * (np.sum(self.params["W1"] ** 2)) + 0.5 * self.reg * (
            np.sum(self.params["W2"] ** 2)
        )
        dout, grads["W2"], grads["b2"] = affine_backward(dout, cache2)
        _, grads["W1"], grads["b1"] = affine_relu_backward(dout, cache1)
        grads["W2"] += self.reg * self.params["W2"]
        grads["W1"] += self.reg * self.params["W1"]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
    ReLU nonlinearities, and a softmax loss function. This will also implement
```

```
    dropout and batch normalization as options. For a network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """

    def __init__(
        self,
        hidden_dims,
        input_dim=3 * 32 * 32,
        num_classes=10,
        dropout=0,
        use_batchnorm=False,
        reg=0.0,
        weight_scale=1e-2,
        dtype=np.float32,
        seed=None,
    ):
        """
        Initialize a new FullyConnectedNet.

        Inputs:
        - hidden_dims: A list of integers giving the size of each hidden layer.
        - input_dim: An integer giving the size of the input.
        - num_classes: An integer giving the number of classes to classify.
        - dropout: Scalar between 0 and 1 giving dropout strength. If dropout=0 then
          the network should not use dropout at all.
        - use_batchnorm: Whether or not the network should use batch normalization.
        - reg: Scalar giving L2 regularization strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - dtype: A numpy datatype object; all computations will be performed using
          this datatype. float32 is faster but less accurate, so you should use
          float64 for numeric gradient checking.
        - seed: If not None, then pass this random seed to the dropout layers. This
          will make the dropout layers deteriminstic so we can gradient check the
          model.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
        self.dtype = dtype
```

```
        self.params = {}

        ############################################################################
        # HW2: Initialize the parameters of the network, storing all values in     #
        # the self.params dictionary. Store weights and biases for the first layer #
        # in W1 and b1; for the second layer use W2 and b2, etc. Weights should be #
        # initialized from a normal distribution with standard deviation equal to  #
        # weight_scale and biases should be initialized to zero.                   #
        #                                                                          #
        # When using batch normalization, store scale and shift parameters for the #
        # first layer in gamma1 and beta1; for the second layer use gamma2 and     #
        # beta2, etc. Scale parameters should be initialized to one and shift      #
        # parameters should be initialized to zero.                                #
        ############################################################################
        for layer in range(self.num_layers - 1):
            dim_input = input_dim if layer == 0 else hidden_dims[layer - 1]

            self.params["W" + str(layer + 1)] = (
                np.random.randn(dim_input, hidden_dims[layer]) * weight_scale
            )
            self.params["b" + str(layer + 1)] = np.zeros(hidden_dims[layer])

            if self.use_batchnorm:
                self.params["gamma" + str(layer + 1)] = np.ones(hidden_dims[layer])
                self.params["beta" + str(layer + 1)] = np.zeros(hidden_dims[layer])

        self.params["W" + str(self.num_layers)] = (
            np.random.randn(hidden_dims[self.num_layers - 2], num_classes)
            * weight_scale
        )
        self.params["b" + str(self.num_layers)] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # When using dropout we need to pass a dropout_param dictionary to each
        # dropout layer so that the layer knows the dropout probability and the mode
        # (train / test). You can pass the same dropout_param to each dropout layer.
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {"mode": "train", "p": dropout}
            if seed is not None:
                self.dropout_param["seed"] = seed

        # With batch normalization we need to keep track of running means and
        # variances, so we need to pass a special bn_param object to each batch
        # normalization layer. You should pass self.bn_params[0] to the forward pass
        # of the first batch normalization layer, self.bn_params[1] to the forward
        # pass of the second batch normalization layer, etc.
```

```
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{"mode": "train"} for i in range(self.num_layers - 1)]

        # Cast all parameters to the correct datatype
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.

        Input / output: Same as TwoLayerNet above.
        """
        X = X.astype(self.dtype)
        mode = "test" if y is None else "train"

        # Set train/test mode for batchnorm params and dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param["mode"] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param[mode] = mode

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the fully-connected net, computing  #
        # the class scores for X and storing them in the scores variable.          #
        #                                                                          #
        # You can use your solution for HW1 and add batch norm and/or dropout      #
        # whenever `self.use_batchnorm` and/or `self.use_dropout` are true         #
        #                                                                          #
        # When using dropout, you'll need to pass self.dropout_param to each       #
        # dropout forward pass.                                                    #
        #                                                                          #
        # When using batch normalization, you'll need to pass self.bn_params[0] to #
        # the forward pass for the first batch normalization layer, pass           #
        # self.bn_params[1] to the forward pass for the second batch normalization #
        # layer, etc.                                                              #
        ############################################################################
        caches = []
        out = X

        # Forward through hidden layers
        for i in range(1, self.num_layers):
            W = self.params['W' + str(i)]
            b = self.params['b' + str(i)]
```

```
            if self.use_batchnorm:
                gamma = self.params['gamma' + str(i)]
                beta = self.params['beta' + str(i)]
                bn_param = self.bn_params[i - 1]
                out, cache = affine_bn_relu_forward(out, W, b, gamma, beta, bn_param)
            else:
                out, cache = affine_relu_forward(out, W, b)
            caches.append(cache)

            if self.use_dropout:
                out, do_cache = dropout_forward(out, self.dropout_param)
                caches.append(do_cache)

        # Final affine layer
        W_final = self.params['W' + str(self.num_layers)]
        b_final = self.params['b' + str(self.num_layers)]
        scores, cache_final = affine_forward(out, W_final, b_final)
        caches.append(cache_final)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # If test mode return early
        if mode == "test":
            return scores

        loss, grads = 0.0, {}
        ############################################################################
        # TODO: Implement the backward pass for the fully-connected net. Store the #
        # loss in the loss variable and gradients in the grads dictionary. Compute #
        # data loss using softmax, and make sure that grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2 regularization on the         #
        # weights, but not the biases.                                             #
        #                                                                          #
        # When using batch normalization, you don't need to regularize the scale   #
        # and shift parameters.                                                    #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        # FC scores -> softmax. Calculate loss and gradients
        loss, dout = softmax_loss(scores, y)

        # Add L2 regularization to loss
        for i in range(1, self.num_layers + 1):
            loss += 0.5 * self.reg * np.sum(self.params['W' + str(i)] ** 2)

        # Backward through final layer
```

```
        cache_final = caches.pop()
        dout, grads['W' + str(self.num_layers)], grads['b' + str(self.num_layers)] =
affine_backward(dout, cache_final)
        grads['W' + str(self.num_layers)] += self.reg * self.params['W' +
str(self.num_layers)]

        # Backward through hidden layers (in reverse order)
        for i in range(self.num_layers - 1, 0, -1):
            if self.use_dropout:
                do_cache = caches.pop()
                dout = dropout_backward(dout, do_cache)

            cache = caches.pop()
            if self.use_batchnorm:
                dout, grads['W' + str(i)], grads['b' + str(i)], grads['gamma' + str(i)],
grads['beta' + str(i)] = affine_bn_relu_backward(dout, cache)
            else:
                dout, grads['W' + str(i)], grads['b' + str(i)] = affine_relu_backward(dout,
cache)
            grads['W' + str(i)] += self.reg * self.params['W' + str(i)]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads
```

Fixing the FullyConnectedNet implementation. Checking available helper functions:

Completing ThreeLayerConvNet:

```
import numpy as np

from deeplearning.layer_utils import *


class ThreeLayerConvNet(object):
    """
    A three-layer convolutional network with the following architecture:

    conv - relu - 2x2 max pool - affine - relu - affine - softmax

    The network operates on minibatches of data that have shape (N, C, H, W)
    consisting of N images, each with height H and width W and with C input
    channels.
    """
```

```
    def __init__(
        self,
        input_dim=(3, 32, 32),
        num_filters=32,
        filter_size=7,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-3,
        reg=0.0,
        dtype=np.float32,
    ):
        """
        Initialize a new network.

        Inputs:
        - input_dim: Tuple (C, H, W) giving size of input data
        - num_filters: Number of filters to use in the convolutional layer
        - filter_size: Size of filters to use in the convolutional layer
        - hidden_dim: Number of units to use in the fully-connected hidden layer
        - num_classes: Number of scores to produce from the final affine layer.
        - weight_scale: Scalar giving standard deviation for random initialization
          of weights.
        - reg: Scalar giving L2 regularization strength
        - dtype: numpy datatype to use for computation.
        """
        self.params = {}
        self.reg = reg
        self.dtype = dtype

        ############################################################################
        # TODO: Initialize weights and biases for the three-layer convolutional    #
        # network. Weights should be initialized from a Gaussian with standard     #
        # deviation equal to weight_scale; biases should be initialized to zero.   #
        # All weights and biases should be stored in the dictionary self.params.   #
        # Store weights and biases for the convolutional layer using the keys 'W1' #
        # and 'b1'; use keys 'W2' and 'b2' for the weights and biases of the       #
        # hidden affine layer, and keys 'W3' and 'b3' for the weights and biases   #
        # of the output affine layer.                                              #
        ############################################################################
        C, H, W = input_dim

        # Conv layer: (num_filters, C, filter_size, filter_size)
        self.params['W1'] = weight_scale * np.random.randn(num_filters, C, filter_size,
filter_size)
        self.params['b1'] = np.zeros(num_filters)

        # After conv+pool: H' = (H + 2*pad - filter_size) // stride + 1, then // 2 for pool
        # Assuming stride=1, pad=(filter_size-1)//2 to maintain size, then pool reduces by 2
        conv_H = H // 2
```

```
        conv_W = W // 2

        # Affine layer input size
        affine_input_size = num_filters * conv_H * conv_W

        # Hidden affine layer
        self.params['W2'] = weight_scale * np.random.randn(affine_input_size, hidden_dim)
        self.params['b2'] = np.zeros(hidden_dim)

        # Output affine layer
        self.params['W3'] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params['b3'] = np.zeros(num_classes)
        ############################################################################
        #                            END OF YOUR CODE                              #
        ############################################################################

        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Evaluate loss and gradient for the three-layer convolutional network.

        Input / output: Same API as TwoLayerNet in fc_net.py.
        """
        W1, b1 = self.params["W1"], self.params["b1"]
        W2, b2 = self.params["W2"], self.params["b2"]
        W3, b3 = self.params["W3"], self.params["b3"]

        # pass conv_param to the forward pass for the convolutional layer
        filter_size = W1.shape[2]
        conv_param = {"stride": 1, "pad": (filter_size - 1) // 2}

        # pass pool_param to the forward pass for the max-pooling layer
        pool_param = {"pool_height": 2, "pool_width": 2, "stride": 2}

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the three-layer convolutional net,  #
        # computing the class scores for X and storing them in the scores          #
        # variable.                                                                #
        ############################################################################

        ############################################################################
        #                            END OF YOUR CODE                              #
        ############################################################################

        if y is None:
            return scores
```

```
        loss, grads = 0, {}
        ############################################################################
        # TODO: Implement the backward pass for the three-layer convolutional net, #
        # storing the loss and gradients in the loss and grads variables. Compute  #
        # data loss using softmax, and make sure that grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2 regularization!               #
        ############################################################################

        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


pass
```

```
import numpy as np

from deeplearning.layer_utils import *


class ThreeLayerConvNet(object):
    """
    A three-layer convolutional network with the following architecture:

    conv - relu - 2x2 max pool - affine - relu - affine - softmax

    The network operates on minibatches of data that have shape (N, C, H, W)
    consisting of N images, each with height H and width W and with C input
    channels.
    """

    def __init__(
        self,
        input_dim=(3, 32, 32),
        num_filters=32,
        filter_size=7,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-3,
        reg=0.0,
        dtype=np.float32,
    ):
        """
        Initialize a new network.
```

```
        Inputs:
        - input_dim: Tuple (C, H, W) giving size of input data
        - num_filters: Number of filters to use in the convolutional layer
        - filter_size: Size of filters to use in the convolutional layer
        - hidden_dim: Number of units to use in the fully-connected hidden layer
        - num_classes: Number of scores to produce from the final affine layer.
        - weight_scale: Scalar giving standard deviation for random initialization
          of weights.
        - reg: Scalar giving L2 regularization strength
        - dtype: numpy datatype to use for computation.
        """
        self.params = {}
        self.reg = reg
        self.dtype = dtype

        ############################################################################
        # TODO: Initialize weights and biases for the three-layer convolutional    #
        # network. Weights should be initialized from a Gaussian with standard     #
        # deviation equal to weight_scale; biases should be initialized to zero.   #
        # All weights and biases should be stored in the dictionary self.params.   #
        # Store weights and biases for the convolutional layer using the keys 'W1' #
        # and 'b1'; use keys 'W2' and 'b2' for the weights and biases of the       #
        # hidden affine layer, and keys 'W3' and 'b3' for the weights and biases   #
        # of the output affine layer.                                              #
        ############################################################################
        C, H, W = input_dim

        # Conv layer: (num_filters, C, filter_size, filter_size)
        self.params['W1'] = weight_scale * np.random.randn(num_filters, C, filter_size,
filter_size)
        self.params['b1'] = np.zeros(num_filters)

        # After conv+pool: H' = (H + 2*pad - filter_size) // stride + 1, then // 2 for pool
        # Assuming stride=1, pad=(filter_size-1)//2 to maintain size, then pool reduces by 2
        conv_H = H // 2
        conv_W = W // 2

        # Affine layer input size
        affine_input_size = num_filters * conv_H * conv_W

        # Hidden affine layer
        self.params['W2'] = weight_scale * np.random.randn(affine_input_size, hidden_dim)
        self.params['b2'] = np.zeros(hidden_dim)

        # Output affine layer
        self.params['W3'] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params['b3'] = np.zeros(num_classes)
        ############################################################################
```

```
        #                             END OF YOUR CODE                             #
        ############################################################################

        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Evaluate loss and gradient for the three-layer convolutional network.

        Input / output: Same API as TwoLayerNet in fc_net.py.
        """
        W1, b1 = self.params["W1"], self.params["b1"]
        W2, b2 = self.params["W2"], self.params["b2"]
        W3, b3 = self.params["W3"], self.params["b3"]

        # pass conv_param to the forward pass for the convolutional layer
        filter_size = W1.shape[2]
        conv_param = {"stride": 1, "pad": (filter_size - 1) // 2}

        # pass pool_param to the forward pass for the max-pooling layer
        pool_param = {"pool_height": 2, "pool_width": 2, "stride": 2}

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the three-layer convolutional net,  #
        # computing the class scores for X and storing them in the scores          #
        # variable.                                                                #
        ############################################################################
        # Conv - ReLU - MaxPool
        conv_out, conv_cache = conv_relu_pool_forward(X, W1, b1, conv_param, pool_param)

        # Affine - ReLU
        conv_out_flat = conv_out.reshape(conv_out.shape[0], -1)
        affine1_out, affine1_cache = affine_relu_forward(conv_out_flat, W2, b2)

        # Affine (output)
        scores, affine2_cache = affine_forward(affine1_out, W3, b3)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # TODO: Implement the backward pass for the three-layer convolutional net, #
        # storing the loss and gradients in the loss and grads variables. Compute  #
```

```
        # data loss using softmax, and make sure that grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2 regularization!               #
        ############################################################################

        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


pass
```

```
import numpy as np

from deeplearning.layer_utils import *


class ThreeLayerConvNet(object):
    """
    A three-layer convolutional network with the following architecture:

    conv - relu - 2x2 max pool - affine - relu - affine - softmax

    The network operates on minibatches of data that have shape (N, C, H, W)
    consisting of N images, each with height H and width W and with C input
    channels.
    """

    def __init__(
        self,
        input_dim=(3, 32, 32),
        num_filters=32,
        filter_size=7,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-3,
        reg=0.0,
        dtype=np.float32,
    ):
        """
        Initialize a new network.

        Inputs:
        - input_dim: Tuple (C, H, W) giving size of input data
        - num_filters: Number of filters to use in the convolutional layer
        - filter_size: Size of filters to use in the convolutional layer
```

```
        - hidden_dim: Number of units to use in the fully-connected hidden layer
        - num_classes: Number of scores to produce from the final affine layer.
        - weight_scale: Scalar giving standard deviation for random initialization
          of weights.
        - reg: Scalar giving L2 regularization strength
        - dtype: numpy datatype to use for computation.
        """
        self.params = {}
        self.reg = reg
        self.dtype = dtype

        ############################################################################
        # TODO: Initialize weights and biases for the three-layer convolutional    #
        # network. Weights should be initialized from a Gaussian with standard     #
        # deviation equal to weight_scale; biases should be initialized to zero.   #
        # All weights and biases should be stored in the dictionary self.params.   #
        # Store weights and biases for the convolutional layer using the keys 'W1' #
        # and 'b1'; use keys 'W2' and 'b2' for the weights and biases of the       #
        # hidden affine layer, and keys 'W3' and 'b3' for the weights and biases   #
        # of the output affine layer.                                              #
        ############################################################################
        C, H, W = input_dim

        # Conv layer: (num_filters, C, filter_size, filter_size)
        self.params['W1'] = weight_scale * np.random.randn(num_filters, C, filter_size,
filter_size)
        self.params['b1'] = np.zeros(num_filters)

        # After conv+pool: H' = (H + 2*pad - filter_size) // stride + 1, then // 2 for pool
        # Assuming stride=1, pad=(filter_size-1)//2 to maintain size, then pool reduces by 2
        conv_H = H // 2
        conv_W = W // 2

        # Affine layer input size
        affine_input_size = num_filters * conv_H * conv_W

        # Hidden affine layer
        self.params['W2'] = weight_scale * np.random.randn(affine_input_size, hidden_dim)
        self.params['b2'] = np.zeros(hidden_dim)

        # Output affine layer
        self.params['W3'] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params['b3'] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)
```

```
def loss(self, X, y=None):
    """
    Evaluate loss and gradient for the three-layer convolutional network.

    Input / output: Same API as TwoLayerNet in fc_net.py.
    """
    W1, b1 = self.params["W1"], self.params["b1"]
    W2, b2 = self.params["W2"], self.params["b2"]
    W3, b3 = self.params["W3"], self.params["b3"]

    # pass conv_param to the forward pass for the convolutional layer
    filter_size = W1.shape[2]
    conv_param = {"stride": 1, "pad": (filter_size - 1) // 2}

    # pass pool_param to the forward pass for the max-pooling layer
    pool_param = {"pool_height": 2, "pool_width": 2, "stride": 2}

    scores = None
    ############################################################################
    # TODO: Implement the forward pass for the three-layer convolutional net,  #
    # computing the class scores for X and storing them in the scores          #
    # variable.                                                                #
    ############################################################################
    # Conv - ReLU - MaxPool
    conv_out, conv_cache = conv_relu_pool_forward(X, W1, b1, conv_param, pool_param)

    # Affine - ReLU
    conv_out_flat = conv_out.reshape(conv_out.shape[0], -1)
    affine1_out, affine1_cache = affine_relu_forward(conv_out_flat, W2, b2)

    # Affine (output)
    scores, affine2_cache = affine_forward(affine1_out, W3, b3)
    ############################################################################
    #                             END OF YOUR CODE                             #
    ############################################################################

    if y is None:
        return scores

    loss, grads = 0, {}
    ############################################################################
    # TODO: Implement the backward pass for the three-layer convolutional net, #
    # storing the loss and gradients in the loss and grads variables. Compute  #
    # data loss using softmax, and make sure that grads[k] holds the gradients #
    # for self.params[k]. Don't forget to add L2 regularization!               #
    ############################################################################
    # Compute loss
    loss, dout = softmax_loss(scores, y)
```

```
        # Add L2 regularization
        loss += 0.5 * self.reg * (np.sum(W1**2) + np.sum(W2**2) + np.sum(W3**2))

        # Backward through affine2
        dout, grads['W3'], grads['b3'] = affine_backward(dout, affine2_cache)
        grads['W3'] += self.reg * W3

        # Backward through affine1
        dout, grads['W2'], grads['b2'] = affine_relu_backward(dout, affine1_cache)
        grads['W2'] += self.reg * W2

        # Reshape for conv backward
        dout = dout.reshape(conv_out.shape)

        # Backward through conv
        dout, grads['W1'], grads['b1'] = conv_relu_pool_backward(dout, conv_cache)
        grads['W1'] += self.reg * W1
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


pass
```

```
import numpy as np

from deeplearning.layer_utils import *
from deeplearning.layers import *


class ThreeLayerConvNet(object):
    """
    A three-layer convolutional network with the following architecture:

    conv - relu - 2x2 max pool - affine - relu - affine - softmax

    The network operates on minibatches of data that have shape (N, C, H, W)
    consisting of N images, each with height H and width W and with C input
    channels.
    """

    def __init__(
        self,
        input_dim=(3, 32, 32),
```

```
        num_filters=32,
        filter_size=7,
        hidden_dim=100,
        num_classes=10,
        weight_scale=1e-3,
        reg=0.0,
        dtype=np.float32,
    ):
        """
        Initialize a new network.

        Inputs:
        - input_dim: Tuple (C, H, W) giving size of input data
        - num_filters: Number of filters to use in the convolutional layer
        - filter_size: Size of filters to use in the convolutional layer
        - hidden_dim: Number of units to use in the fully-connected hidden layer
        - num_classes: Number of scores to produce from the final affine layer.
        - weight_scale: Scalar giving standard deviation for random initialization
          of weights.
        - reg: Scalar giving L2 regularization strength
        - dtype: numpy datatype to use for computation.
        """
        self.params = {}
        self.reg = reg
        self.dtype = dtype

        ############################################################################
        # TODO: Initialize weights and biases for the three-layer convolutional    #
        # network. Weights should be initialized from a Gaussian with standard     #
        # deviation equal to weight_scale; biases should be initialized to zero.   #
        # All weights and biases should be stored in the dictionary self.params.   #
        # Store weights and biases for the convolutional layer using the keys 'W1' #
        # and 'b1'; use keys 'W2' and 'b2' for the weights and biases of the       #
        # hidden affine layer, and keys 'W3' and 'b3' for the weights and biases   #
        # of the output affine layer.                                              #
        ############################################################################
        C, H, W = input_dim

        # Conv layer: (num_filters, C, filter_size, filter_size)
        self.params['W1'] = weight_scale * np.random.randn(num_filters, C, filter_size,
filter_size)
        self.params['b1'] = np.zeros(num_filters)

        # After conv+pool: H' = (H + 2*pad - filter_size) // stride + 1, then // 2 for pool
        # Assuming stride=1, pad=(filter_size-1)//2 to maintain size, then pool reduces by 2
        conv_H = H // 2
        conv_W = W // 2

        # Affine layer input size
```

```
        affine_input_size = num_filters * conv_H * conv_W

        # Hidden affine layer
        self.params['W2'] = weight_scale * np.random.randn(affine_input_size, hidden_dim)
        self.params['b2'] = np.zeros(hidden_dim)

        # Output affine layer
        self.params['W3'] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params['b3'] = np.zeros(num_classes)
        ############################################################################
        #                            END OF YOUR CODE                              #
        ############################################################################

        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Evaluate loss and gradient for the three-layer convolutional network.

        Input / output: Same API as TwoLayerNet in fc_net.py.
        """
        W1, b1 = self.params["W1"], self.params["b1"]
        W2, b2 = self.params["W2"], self.params["b2"]
        W3, b3 = self.params["W3"], self.params["b3"]

        # pass conv_param to the forward pass for the convolutional layer
        filter_size = W1.shape[2]
        conv_param = {"stride": 1, "pad": (filter_size - 1) // 2}

        # pass pool_param to the forward pass for the max-pooling layer
        pool_param = {"pool_height": 2, "pool_width": 2, "stride": 2}

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the three-layer convolutional net,  #
        # computing the class scores for X and storing them in the scores          #
        # variable.                                                                #
        ############################################################################
        # Conv - ReLU - MaxPool
        conv_out, conv_cache = conv_relu_pool_forward(X, W1, b1, conv_param, pool_param)

        # Affine - ReLU
        conv_out_flat = conv_out.reshape(conv_out.shape[0], -1)
        affine1_out, affine1_cache = affine_relu_forward(conv_out_flat, W2, b2)

        # Affine (output)
        scores, affine2_cache = affine_forward(affine1_out, W3, b3)
        ############################################################################
```

```
        #                             END OF YOUR CODE                             #
        ############################################################################

        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # TODO: Implement the backward pass for the three-layer convolutional net, #
        # storing the loss and gradients in the loss and grads variables. Compute  #
        # data loss using softmax, and make sure that grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2 regularization!               #
        ############################################################################
        # Compute loss
        loss, dout = softmax_loss(scores, y)

        # Add L2 regularization
        loss += 0.5 * self.reg * (np.sum(W1**2) + np.sum(W2**2) + np.sum(W3**2))

        # Backward through affine2
        dout, grads['W3'], grads['b3'] = affine_backward(dout, affine2_cache)
        grads['W3'] += self.reg * W3

        # Backward through affine1
        dout, grads['W2'], grads['b2'] = affine_relu_backward(dout, affine1_cache)
        grads['W2'] += self.reg * W2

        # Reshape for conv backward
        dout = dout.reshape(conv_out.shape)

        # Backward through conv
        dout, grads['W1'], grads['b1'] = conv_relu_pool_backward(dout, conv_cache)
        grads['W1'] += self.reg * W1
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads


pass
```

Completing the PyTorch NeuralNetwork model:

```
import torch
import torch.nn as nn
```

```
class NeuralNetwork(nn.Module):
    def __init__(self, do_batchnorm=False, p_dropout=0.0):
        super().__init__()
        self.do_batchnorm = do_batchnorm
        self.p_dropout = p_dropout

        ############################################################################
        # TODO: Complete the implementation of the first convolutional block
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        # Refer to pytorch.org for API documentations.
        #
        # The first conv block consists of a conv layer, an optional spatial
        #  batch normalization layer, a ReLU activation layer, and a maxpooling
        #  layer:
        # [conv1] -> ([bn1]) -> [relu1] -> [pool1]
        #
        # All conv layers in this neural network uses a kernel size of 3 and a
        #  padding of 1.
        #
        # Batch norm is enabled if and only if `do_batchnorm` is true
        #
        # All max-pooling layers in this neural network pools each non-
        #  overlapping 2x2 patch to a single pixel, shrinking the height/width
        #  of the feature map by 1/2.
        #
        # The first conv block has 16 filters
        ############################################################################
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2, 2)
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Implement the second convolutional block
        #
        # The second convolutional block has the same structure as the first,
        #  except that the conv layer has 32 filters
        ############################################################################

        ############################################################################
        # End of your code
        ############################################################################
```

```
        ############################################################################
        # TODO: Implement the third convolutional block
        #
        # The third convolutional block uses a strided conv layer with a stride
        #  of 2. It has 64 filters
        #
        # The conv layer is followed by an optional spatial batch norm layer,
        #  and a ReLU activation layer. No pooling in this block.
        ############################################################################

        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Complete the 2-layer fully-connected classifier
        #
        # The input to this classifier is a flattened 1024-d vector for each
        #  input image.
        #
        # The classifier consists of two fully-connected layers, with a hidden
        #  dimension of 256 and an output dimension of 100. Dropout after the
        #  activation layer is enabled if and only if `p_dropout > 0.0`
        # [fc1] -> [relu4] -> ([drop]) -> [fc2]
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        ############################################################################

        ############################################################################
        # End of your code
        ############################################################################

    def forward(self, x):
        # The shape of `x` is [bsz, 3, 32, 32]

        x = self.conv1(x)  # [bsz, 16, 32, 32]
        if self.do_batchnorm:
            x = self.bn1(x)
        x = self.pool1(self.relu1(x))  # [bsz, 16, 16, 16]

        x = self.conv2(x)  # [bsz, 32, 16, 16]
        if self.do_batchnorm:
            x = self.bn2(x)
        x = self.pool2(self.relu2(x))  # [bsz, 32, 8, 8]

        x = self.conv3(x)  # [bsz, 64, 4, 4]
        if self.do_batchnorm:
```

```
            x = self.bn3(x)
        x = self.relu3(x)

        x = torch.flatten(x, 1)  # [bsz, 1024]
        x = self.relu4(self.fc1(x))  # [bsz, 256]
        if self.p_dropout > 0.0:
            x = self.drop(x)
        x = self.fc2(x)  # [bsz, 100]
        return x
```

```
import torch
import torch.nn as nn


class NeuralNetwork(nn.Module):
    def __init__(self, do_batchnorm=False, p_dropout=0.0):
        super().__init__()
        self.do_batchnorm = do_batchnorm
        self.p_dropout = p_dropout

        ########################################################################
        # TODO: Complete the implementation of the first convolutional block
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        # Refer to pytorch.org for API documentations.
        #
        # The first conv block consists of a conv layer, an optional spatial
        #  batch normalization layer, a ReLU activation layer, and a maxpooling
        #  layer:
        # [conv1] -> ([bn1]) -> [relu1] -> [pool1]
        #
        # All conv layers in this neural network uses a kernel size of 3 and a
        #  padding of 1.
        #
        # Batch norm is enabled if and only if `do_batchnorm` is true
        #
        # All max-pooling layers in this neural network pools each non-
        #  overlapping 2x2 patch to a single pixel, shrinking the height/width
        #  of the feature map by 1/2.
        #
        # The first conv block has 16 filters
        ########################################################################
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()
```

```
        self.pool1 = nn.MaxPool2d(2, 2)
        ########################################################################
        # End of your code
        ########################################################################

        ########################################################################
        # TODO: Implement the second convolutional block
        #
        # The second convolutional block has the same structure as the first,
        #  except that the conv layer has 32 filters
        ########################################################################
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn2 = nn.BatchNorm2d(32)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2, 2)
        ########################################################################
        # End of your code
        ########################################################################

        ########################################################################
        # TODO: Implement the third convolutional block
        #
        # The third convolutional block uses a strided conv layer with a stride
        #  of 2. It has 64 filters
        #
        # The conv layer is followed by an optional spatial batch norm layer,
        #  and a ReLU activation layer. No pooling in this block.
        ########################################################################

        ########################################################################
        # End of your code
        ########################################################################

        ########################################################################
        # TODO: Complete the 2-layer fully-connected classifier
        #
        # The input to this classifier is a flattened 1024-d vector for each
        #  input image.
        #
        # The classifier consists of two fully-connected layers, with a hidden
        #  dimension of 256 and an output dimension of 100. Dropout after the
        #  activation layer is enabled if and only if `p_dropout > 0.0`
        # [fc1] -> [relu4] -> ([drop]) -> [fc2]
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        ########################################################################
```

```
        ##########################################################################
        # End of your code
        ##########################################################################

    def forward(self, x):
        # The shape of `x` is [bsz, 3, 32, 32]

        x = self.conv1(x)  # [bsz, 16, 32, 32]
        if self.do_batchnorm:
            x = self.bn1(x)
        x = self.pool1(self.relu1(x))  # [bsz, 16, 16, 16]

        x = self.conv2(x)  # [bsz, 32, 16, 16]
        if self.do_batchnorm:
            x = self.bn2(x)
        x = self.pool2(self.relu2(x))  # [bsz, 32, 8, 8]

        x = self.conv3(x)  # [bsz, 64, 4, 4]
        if self.do_batchnorm:
            x = self.bn3(x)
        x = self.relu3(x)

        x = torch.flatten(x, 1)  # [bsz, 1024]
        x = self.relu4(self.fc1(x))  # [bsz, 256]
        if self.p_dropout > 0.0:
            x = self.drop(x)
        x = self.fc2(x)  # [bsz, 100]
        return x
```

```
import torch
import torch.nn as nn


class NeuralNetwork(nn.Module):
    def __init__(self, do_batchnorm=False, p_dropout=0.0):
        super().__init__()
        self.do_batchnorm = do_batchnorm
        self.p_dropout = p_dropout

        ##########################################################################
        # TODO: Complete the implementation of the first convolutional block
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        # Refer to pytorch.org for API documentations.
        #
        # The first conv block consists of a conv layer, an optional spatial
```

```
        #  batch normalization layer, a ReLU activation layer, and a maxpooling
        #  layer:
        # [conv1] -> ([bn1]) -> [relu1] -> [pool1]
        #
        # All conv layers in this neural network uses a kernel size of 3 and a
        #  padding of 1.
        #
        # Batch norm is enabled if and only if `do_batchnorm` is true
        #
        # All max-pooling layers in this neural network pools each non-
        #  overlapping 2x2 patch to a single pixel, shrinking the height/width
        #  of the feature map by 1/2.
        #
        # The first conv block has 16 filters
        ############################################################################
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2, 2)
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Implement the second convolutional block
        #
        # The second convolutional block has the same structure as the first,
        #  except that the conv layer has 32 filters
        ############################################################################
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn2 = nn.BatchNorm2d(32)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2, 2)
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Implement the third convolutional block
        #
        # The third convolutional block uses a strided conv layer with a stride
        #  of 2. It has 64 filters
        #
        # The conv layer is followed by an optional spatial batch norm layer,
        #  and a ReLU activation layer. No pooling in this block.
        ############################################################################
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1, stride=2)
```

```
        if self.do_batchnorm:
            self.bn3 = nn.BatchNorm2d(64)
        self.relu3 = nn.ReLU()
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Complete the 2-layer fully-connected classifier
        #
        # The input to this classifier is a flattened 1024-d vector for each
        #  input image.
        #
        # The classifier consists of two fully-connected layers, with a hidden
        #  dimension of 256 and an output dimension of 100. Dropout after the
        #  activation layer is enabled if and only if `p_dropout > 0.0`
        # [fc1] -> [relu4] -> ([drop]) -> [fc2]
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        ############################################################################

        ############################################################################
        # End of your code
        ############################################################################

    def forward(self, x):
        # The shape of `x` is [bsz, 3, 32, 32]

        x = self.conv1(x)  # [bsz, 16, 32, 32]
        if self.do_batchnorm:
            x = self.bn1(x)
        x = self.pool1(self.relu1(x))  # [bsz, 16, 16, 16]

        x = self.conv2(x)  # [bsz, 32, 16, 16]
        if self.do_batchnorm:
            x = self.bn2(x)
        x = self.pool2(self.relu2(x))  # [bsz, 32, 8, 8]

        x = self.conv3(x)  # [bsz, 64, 4, 4]
        if self.do_batchnorm:
            x = self.bn3(x)
        x = self.relu3(x)

        x = torch.flatten(x, 1)  # [bsz, 1024]
        x = self.relu4(self.fc1(x))  # [bsz, 256]
        if self.p_dropout > 0.0:
            x = self.drop(x)
        x = self.fc2(x)  # [bsz, 100]
```

```
        return x
```

```
import torch
import torch.nn as nn


class NeuralNetwork(nn.Module):
    def __init__(self, do_batchnorm=False, p_dropout=0.0):
        super().__init__()
        self.do_batchnorm = do_batchnorm
        self.p_dropout = p_dropout

        ############################################################################
        # TODO: Complete the implementation of the first convolutional block
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        # Refer to pytorch.org for API documentations.
        #
        # The first conv block consists of a conv layer, an optional spatial
        #  batch normalization layer, a ReLU activation layer, and a maxpooling
        #  layer:
        # [conv1] -> ([bn1]) -> [relu1] -> [pool1]
        #
        # All conv layers in this neural network uses a kernel size of 3 and a
        #  padding of 1.
        #
        # Batch norm is enabled if and only if `do_batchnorm` is true
        #
        # All max-pooling layers in this neural network pools each non-
        #  overlapping 2x2 patch to a single pixel, shrinking the height/width
        #  of the feature map by 1/2.
        #
        # The first conv block has 16 filters
        ############################################################################
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn1 = nn.BatchNorm2d(16)
        self.relu1 = nn.ReLU()
        self.pool1 = nn.MaxPool2d(2, 2)
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Implement the second convolutional block
        #
```

```
        # The second convolutional block has the same structure as the first,
        #  except that the conv layer has 32 filters
        ############################################################################
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        if self.do_batchnorm:
            self.bn2 = nn.BatchNorm2d(32)
        self.relu2 = nn.ReLU()
        self.pool2 = nn.MaxPool2d(2, 2)
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Implement the third convolutional block
        #
        # The third convolutional block uses a strided conv layer with a stride
        #  of 2. It has 64 filters
        #
        # The conv layer is followed by an optional spatial batch norm layer,
        #  and a ReLU activation layer. No pooling in this block.
        ############################################################################
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1, stride=2)
        if self.do_batchnorm:
            self.bn3 = nn.BatchNorm2d(64)
        self.relu3 = nn.ReLU()
        ############################################################################
        # End of your code
        ############################################################################

        ############################################################################
        # TODO: Complete the 2-layer fully-connected classifier
        #
        # The input to this classifier is a flattened 1024-d vector for each
        #  input image.
        #
        # The classifier consists of two fully-connected layers, with a hidden
        #  dimension of 256 and an output dimension of 100. Dropout after the
        #  activation layer is enabled if and only if `p_dropout > 0.0`
        # [fc1] -> [relu4] -> ([drop]) -> [fc2]
        #
        # Feel free to check the hints of tensor shapes in the `forward` method
        #
        ############################################################################
        self.fc1 = nn.Linear(1024, 256)
        self.relu4 = nn.ReLU()
        if self.p_dropout > 0.0:
            self.drop = nn.Dropout(self.p_dropout)
        self.fc2 = nn.Linear(256, 100)
        ############################################################################
```

```
        # End of your code
        ##########################################################################

    def forward(self, x):
        # The shape of `x` is [bsz, 3, 32, 32]

        x = self.conv1(x)  # [bsz, 16, 32, 32]
        if self.do_batchnorm:
            x = self.bn1(x)
        x = self.pool1(self.relu1(x))  # [bsz, 16, 16, 16]

        x = self.conv2(x)  # [bsz, 32, 16, 16]
        if self.do_batchnorm:
            x = self.bn2(x)
        x = self.pool2(self.relu2(x))  # [bsz, 32, 8, 8]

        x = self.conv3(x)  # [bsz, 64, 4, 4]
        if self.do_batchnorm:
            x = self.bn3(x)
        x = self.relu3(x)

        x = torch.flatten(x, 1)  # [bsz, 1024]
        x = self.relu4(self.fc1(x))  # [bsz, 256]
        if self.p_dropout > 0.0:
            x = self.drop(x)
        x = self.fc2(x)  # [bsz, 100]
        return x
```

Completing the train function in the PyTorch notebook:

Searching for the train function:

Completing the train function in the notebook:

Searching for the exact content to edit:

Editing the notebook cell to complete the train function:

```
        ##########################################################################
        # TODO: complete the following code for backpropagation and gradient
        #  update of a single step.
```

```
        # Hint: 3 lines
        ############################################################################
        raise NotImplementedError()
        ############################################################################

->

        ############################################################################
        # TODO: complete the following code for backpropagation and gradient
        #  update of a single step.
        # Hint: 3 lines
        ############################################################################
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        ############################################################################
```

Fixing a bug in FullyConnectedNet: storing caches correctly:

Fixing the bn_param mode setting:

```
import numpy as np

from deeplearning.layer_utils import *
from deeplearning.layers import *


class TwoLayerNet(object):
    """
    A two-layer fully-connected neural network with ReLU nonlinearity and
    softmax loss that uses a modular layer design. We assume an input dimension
    of D, a hidden dimension of H, and perform classification over C classes.

    The architecure should be affine - relu - affine - softmax.

    Note that this class does not implement gradient descent; instead, it
    will interact with a separate Solver object that is responsible for running
    optimization.

    The learnable parameters of the model are stored in the dictionary
    self.params that maps parameter names to numpy arrays.
    """

    def __init__(
        self,
        input_dim=3 * 32 * 32,
        hidden_dim=100,
```

```
        num_classes=10,
        weight_scale=1e-2,
        reg=0.0,
    ):
        """
        Initialize a new network.

        Inputs:
        - input_dim: An integer giving the size of the input
        - hidden_dim: An integer giving the size of the hidden layer
        - num_classes: An integer giving the number of classes to classify
        - dropout: Scalar between 0 and 1 giving dropout strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - reg: Scalar giving L2 regularization strength.
        """
        self.params = {}
        self.reg = reg

        ############################################################################
        # HW1: Initialize the weights and biases of the two-layer net. Weights    #
        # should be initialized from a Gaussian with standard deviation equal to   #
        # weight_scale, and biases should be initialized to zero. All weights and  #
        # biases should be stored in the dictionary self.params, with first layer  #
        # weights and biases using the keys 'W1' and 'b1' and second layer weights #
        # and biases using the keys 'W2' and 'b2'.                                 #
        ############################################################################
        self.params["W1"] = weight_scale * np.random.randn(input_dim, hidden_dim)
        self.params["b1"] = np.zeros(hidden_dim)
        self.params["W2"] = weight_scale * np.random.randn(hidden_dim, num_classes)
        self.params["b2"] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

    def loss(self, X, y=None):
        """
        Compute loss and gradient for a minibatch of data.

        Inputs:
        - X: Array of input data of shape (N, d_1, ..., d_k)
        - y: Array of labels, of shape (N,). y[i] gives the label for X[i].

        Returns:
        If y is None, then run a test-time forward pass of the model and return:
        - scores: Array of shape (N, C) giving classification scores, where
          scores[i, c] is the classification score for X[i] and class c.

        If y is not None, then run a training-time forward and backward pass and
```

```
        return a tuple of:
        - loss: Scalar value giving the loss
        - grads: Dictionary with the same keys as self.params, mapping parameter
          names to gradients of the loss with respect to those parameters.
        """
        scores = None
        ############################################################################
        # HW1: Implement the forward pass for the two-layer net, computing the    #
        # class scores for X and storing them in the scores variable.              #
        ############################################################################
        layer1_out, cache1 = affine_relu_forward(
            X, self.params["W1"], self.params["b1"]
        )
        layer2_out, cache2 = affine_forward(
            layer1_out, self.params["W2"], self.params["b2"]
        )
        scores = layer2_out
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # If y is None then we are in test mode so just return scores
        if y is None:
            return scores

        loss, grads = 0, {}
        ############################################################################
        # HW1: Implement the backward pass for the two-layer net. Store the loss  #
        # in the loss variable and gradients in the grads dictionary. Compute data #
        # loss using softmax, and make sure that grads[k] holds the gradients for  #
        # self.params[k]. Don't forget to add L2 regularization on the weights,    #
        # but not the biases.                                                      #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        loss, dout = softmax_loss(scores, y)
        loss += 0.5 * self.reg * (np.sum(self.params["W1"] ** 2)) + 0.5 * self.reg * (
            np.sum(self.params["W2"] ** 2)
        )
        dout, grads["W2"], grads["b2"] = affine_backward(dout, cache2)
        _, grads["W1"], grads["b1"] = affine_relu_backward(dout, cache1)
        grads["W2"] += self.reg * self.params["W2"]
        grads["W1"] += self.reg * self.params["W1"]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################
```

```
        return loss, grads


class FullyConnectedNet(object):
    """
    A fully-connected neural network with an arbitrary number of hidden layers,
    ReLU nonlinearities, and a softmax loss function. This will also implement
    dropout and batch normalization as options. For a network with L layers,
    the architecture will be

    {affine - [batch norm] - relu - [dropout]} x (L - 1) - affine - softmax

    where batch normalization and dropout are optional, and the {...} block is
    repeated L - 1 times.

    Similar to the TwoLayerNet above, learnable parameters are stored in the
    self.params dictionary and will be learned using the Solver class.
    """

    def __init__(
        self,
        hidden_dims,
        input_dim=3 * 32 * 32,
        num_classes=10,
        dropout=0,
        use_batchnorm=False,
        reg=0.0,
        weight_scale=1e-2,
        dtype=np.float32,
        seed=None,
    ):
        """
        Initialize a new FullyConnectedNet.

        Inputs:
        - hidden_dims: A list of integers giving the size of each hidden layer.
        - input_dim: An integer giving the size of the input.
        - num_classes: An integer giving the number of classes to classify.
        - dropout: Scalar between 0 and 1 giving dropout strength. If dropout=0 then
          the network should not use dropout at all.
        - use_batchnorm: Whether or not the network should use batch normalization.
        - reg: Scalar giving L2 regularization strength.
        - weight_scale: Scalar giving the standard deviation for random
          initialization of the weights.
        - dtype: A numpy datatype object; all computations will be performed using
          this datatype. float32 is faster but less accurate, so you should use
          float64 for numeric gradient checking.
        - seed: If not None, then pass this random seed to the dropout layers. This
          will make the dropout layers deteriminstic so we can gradient check the
```

```
        model.
        """
        self.use_batchnorm = use_batchnorm
        self.use_dropout = dropout > 0
        self.reg = reg
        self.num_layers = 1 + len(hidden_dims)
        self.dtype = dtype
        self.params = {}

        ############################################################################
        # HW2: Initialize the parameters of the network, storing all values in     #
        # the self.params dictionary. Store weights and biases for the first layer #
        # in W1 and b1; for the second layer use W2 and b2, etc. Weights should be #
        # initialized from a normal distribution with standard deviation equal to  #
        # weight_scale and biases should be initialized to zero.                   #
        #                                                                          #
        # When using batch normalization, store scale and shift parameters for the #
        # first layer in gamma1 and beta1; for the second layer use gamma2 and     #
        # beta2, etc. Scale parameters should be initialized to one and shift      #
        # parameters should be initialized to zero.                                #
        ############################################################################
        for layer in range(self.num_layers - 1):
            dim_input = input_dim if layer == 0 else hidden_dims[layer - 1]

            self.params["W" + str(layer + 1)] = (
                np.random.randn(dim_input, hidden_dims[layer]) * weight_scale
            )
            self.params["b" + str(layer + 1)] = np.zeros(hidden_dims[layer])

            if self.use_batchnorm:
                self.params["gamma" + str(layer + 1)] = np.ones(hidden_dims[layer])
                self.params["beta" + str(layer + 1)] = np.zeros(hidden_dims[layer])

        self.params["W" + str(self.num_layers)] = (
            np.random.randn(hidden_dims[self.num_layers - 2], num_classes)
            * weight_scale
        )
        self.params["b" + str(self.num_layers)] = np.zeros(num_classes)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # When using dropout we need to pass a dropout_param dictionary to each
        # dropout layer so that the layer knows the dropout probability and the mode
        # (train / test). You can pass the same dropout_param to each dropout layer.
        self.dropout_param = {}
        if self.use_dropout:
            self.dropout_param = {"mode": "train", "p": dropout}
            if seed is not None:
```

```
            self.dropout_param["seed"] = seed

        # With batch normalization we need to keep track of running means and
        # variances, so we need to pass a special bn_param object to each batch
        # normalization layer. You should pass self.bn_params[0] to the forward pass
        # of the first batch normalization layer, self.bn_params[1] to the forward
        # pass of the second batch normalization layer, etc.
        self.bn_params = []
        if self.use_batchnorm:
            self.bn_params = [{"mode": "train"} for i in range(self.num_layers - 1)]

        # Cast all parameters to the correct datatype
        for k, v in self.params.items():
            self.params[k] = v.astype(dtype)

    def loss(self, X, y=None):
        """
        Compute loss and gradient for the fully-connected net.

        Input / output: Same as TwoLayerNet above.
        """
        X = X.astype(self.dtype)
        mode = "test" if y is None else "train"

        # Set train/test mode for batchnorm params and dropout param since they
        # behave differently during training and testing.
        if self.dropout_param is not None:
            self.dropout_param["mode"] = mode
        if self.use_batchnorm:
            for bn_param in self.bn_params:
                bn_param["mode"] = mode

        scores = None
        ############################################################################
        # TODO: Implement the forward pass for the fully-connected net, computing  #
        # the class scores for X and storing them in the scores variable.          #
        #                                                                          #
        # You can use your solution for HW1 and add batch norm and/or dropout      #
        # whenever `self.use_batchnorm` and/or `self.use_dropout` are true         #
        #                                                                          #
        # When using dropout, you'll need to pass self.dropout_param to each       #
        # dropout forward pass.                                                    #
        #                                                                          #
        # When using batch normalization, you'll need to pass self.bn_params[0] to #
        # the forward pass for the first batch normalization layer, pass           #
        # self.bn_params[1] to the forward pass for the second batch normalization #
        # layer, etc.                                                              #
        ############################################################################
        caches = []
```

```
        out = X

        # Forward through hidden layers
        for i in range(1, self.num_layers):
            W = self.params['W' + str(i)]
            b = self.params['b' + str(i)]

            if self.use_batchnorm:
                gamma = self.params['gamma' + str(i)]
                beta = self.params['beta' + str(i)]
                bn_param = self.bn_params[i - 1]
                out, cache = affine_bn_relu_forward(out, W, b, gamma, beta, bn_param)
            else:
                out, cache = affine_relu_forward(out, W, b)
            caches.append(cache)

            if self.use_dropout:
                out, do_cache = dropout_forward(out, self.dropout_param)
                caches.append(do_cache)

        # Final affine layer
        W_final = self.params['W' + str(self.num_layers)]
        b_final = self.params['b' + str(self.num_layers)]
        scores, cache_final = affine_forward(out, W_final, b_final)
        caches.append(cache_final)
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        # If test mode return early
        if mode == "test":
            return scores

        loss, grads = 0.0, {}
        ############################################################################
        # TODO: Implement the backward pass for the fully-connected net. Store the #
        # loss in the loss variable and gradients in the grads dictionary. Compute #
        # data loss using softmax, and make sure that grads[k] holds the gradients #
        # for self.params[k]. Don't forget to add L2 regularization on the         #
        # weights, but not the biases.                                             #
        #                                                                          #
        # When using batch normalization, you don't need to regularize the scale   #
        # and shift parameters.                                                    #
        #                                                                          #
        # NOTE: To ensure that your implementation matches ours and you pass the   #
        # automated tests, make sure that your L2 regularization includes a factor #
        # of 0.5 to simplify the expression for the gradient.                      #
        ############################################################################
        # FC scores -> softmax. Calculate loss and gradients
```

```
        loss, dout = softmax_loss(scores, y)

        # Add L2 regularization to loss
        for i in range(1, self.num_layers + 1):
            loss += 0.5 * self.reg * np.sum(self.params['W' + str(i)] ** 2)

        # Backward through final layer
        cache_final = caches.pop()
        dout, grads['W' + str(self.num_layers)], grads['b' + str(self.num_layers)] =
affine_backward(dout, cache_final)
        grads['W' + str(self.num_layers)] += self.reg * self.params['W' +
str(self.num_layers)]

        # Backward through hidden layers (in reverse order)
        for i in range(self.num_layers - 1, 0, -1):
            if self.use_dropout:
                do_cache = caches.pop()
                dout = dropout_backward(dout, do_cache)

            cache = caches.pop()
            if self.use_batchnorm:
                dout, grads['W' + str(i)], grads['b' + str(i)], grads['gamma' + str(i)],
grads['beta' + str(i)] = affine_bn_relu_backward(dout, cache)
            else:
                dout, grads['W' + str(i)], grads['b' + str(i)] = affine_relu_backward(dout,
cache)
            grads['W' + str(i)] += self.reg * self.params['W' + str(i)]
        ############################################################################
        #                             END OF YOUR CODE                             #
        ############################################################################

        return loss, grads
```

Updating the TODO list:

Checking for linting errors:

Summary of Implementations

1. Batch Normalization (`layers.py`)

- `batchnorm_forward`: Computes mean/variance, normalizes, scales/shifts with gamma/beta, updates running statistics

- `batchnorm_backward`: Standard backward pass through the computation graph
- `batchnorm_backward_alt`: Simplified backward pass using a compact formula

2. Dropout (`layers.py`)

- `dropout_forward`: Inverted dropout during training (scales by 1/(1-p)), no dropout during test
- `dropout_backward`: Backward pass through dropout mask

3. Convolution (`layers.py`)

- `conv_forward_naive`: Naive convolution with padding and stride
- `conv_backward_naive`: Backward pass computing gradients w.r.t. input, weights, and biases

4. Max Pooling (`layers.py`)

- `max_pool_forward_naive`: Max pooling forward pass
- `max_pool_backward_naive`: Backward pass distributing gradients only to max positions

5. Spatial Batch Normalization (`layers.py`)

- `spatial_batchnorm_forward`: Reshapes to use vanilla batch norm, normalizes over spatial dimensions
- `spatial_batchnorm_backward`: Backward pass using vanilla batch norm

6. FullyConnectedNet (`fc_net.py`)

- Completed `loss` method: Forward pass with optional batch norm and dropout, backward pass with L2 regularization

7. ThreeLayerConvNet (`cnn.py`)

- Completed `__init__`: Initializes conv and affine layer weights
- Completed `loss` method: Forward and backward passes through conv-relu-pool and affine layers

8. PyTorch NeuralNetwork (`model.py`)

- Implemented three convolutional blocks with optional batch normalization
- Implemented 2-layer fully-connected classifier with optional dropout
- Completed train function in notebook

All implementations follow the specifications in the notebooks. The code should be ready to run and test. For questions (a)-(d), you'll need to:

- (a) Draw the computational graph (conceptual)

- (b) Derive the closed-form backpropagation (mathematical derivation)
- (c) Run the dropout experiment and analyze results
- (d) Design and tune your own network architecture

The implementations are complete and ready for testing.